

durexForth

Operators Manual

Johan Kotlinski, Kevin Lee Reno, Poindexter Frink, Richard Halkyard

2026-07-16

This manual is for durexForth, a modern Forth system for the Commodore 64.

Table of Contents

Introduction	5
Why Forth?	5
Comparing to other Forths	5
Package Contents	5
Tutorial	7
Meet the Interpreter	7
Introducing the Editor	7
Assembler	7
Console I/O Example	8
Printer Example	8
Editor	9
Inserting Text	9
Navigation	9
Saving and Quitting	11
Text Manipulation	11
Forth Words	13
Stack Manipulation	13
Utility	14
Mathematics	15
Double	17
Float	18
Logic	19
Memory	20
Compiling	20
Word List	22
Variables	23
Control Flow	23
Input	25
Strings	26
Number Formatting	28
Vectored Execution	30
Exceptions	30
Debugging	31
Disk I/O	31
Protocol-Level IEC Device I/O	35
Compatibility	37
Kernel Calls	38
Turn-key Utilities	38

Graphics	40
Turtle Graphics.....	40
High-Resolution Graphics	40
Software Sprites (blitter)	41
Transforming Sprites	43
SID	44
Voice Control.....	44
SID Control.....	44
Reading the SID	44
Music	46
Music Macro Language.....	46
Games and Interrupts	47
Interrupts.....	47
Raster Interrupts	48
Split Screen	49
Background Music	51
Joystick	51
Mouse	52
Sprite Helpers	53
Keyboard	53
Characters and Tiles	55
Scrolling	57
Eight-Way Scrolling	59
VIC-II Register Constants	61
Example: a bouncing ball.....	61
Example: raster colour bars	61
Importing Graphics and Music	62
The catch that bites everyone: where it goes	62
The other catch: loadb eats two bytes.....	62
CharPad Pro.....	62
SpritePad Pro	63
GoatTracker 2	63
EasyFlash Cartridge	65
What is resident.....	65
Including from flash	65
Memory	65
Building the cartridge	66
Assembler.....	67
Variables.....	67
Branches.....	68
Labels	68

Configuring durexForth	70
Stripping Modules	70
Custom Start-Up	70
Turn-key Operation	71
Appendix A: Assembler Mnemonics	72
Appendix B: Memory Map	73
Appendix C: System Exceptions	75
Appendix D: Word Anatomy	76
Appendix E: Avoiding Stack Crashes	77
Appendix F: Internet Resources	78
Appendix G: License	79
Word Index	81

Introduction

Why Forth?

Forth is a unique language. What is so special about it? It is a small, low-level language, which can easily be extended to a high-level, domain-specific language that does anything you want it to. Compared to C64 Basic, Forth is more attractive in almost every way. It is a lot faster, more memory effective, and more powerful.

Compared to C, the nice thing about Forth is that you can run the full development environment on your C64, with text editor, compiler, assembler and debugger. It makes for a more interactive and fun experience than running a cross-compiler on PC.

For a Forth introduction, refer to the excellent [Starting Forth](#) by Leo Brodie.

Comparing to other Forths

There are other Forths for C64, most notably Blazin' Forth. Blazin' Forth is excellent, but durexForth has some advantages:

- durexForth uses text files instead of a custom block file system.
- durexForth is smaller.
- durexForth is faster.
- durexForth can boot from cartridge.
- durexForth implements the Forth 2012 core standard.
- The durexForth editor is a vi clone.
- durexForth is open source (available at [Github](#)).

Package Contents

durexForth is packaged as a 16-kByte .crt cartridge file and a .d64 disk image. Booting from cartridge is equivalent to booting from disk, except that cartridge is faster. The disk contains various optional Forth modules, as well as some appetizer demonstration programs, as follows:

Graphics

The gfxdemo package demonstrates the high-resolution graphics, with some examples adapted from the book "Step-By-Step Programming C64 Graphics" by Phil Cornes. Show the demos by entering:

```
include gfxdemo
```

When an image has finished drawing, press space to continue.

Fractals

The fractals package demonstrates turtle graphics by generating fractal images. Run it by entering:

```
include fractals  
demo
```

When an image has finished drawing, press space to continue.

Music

The mmldemo package demonstrates the MML music capabilities. To play some music:

```
include mmldemo
```

Sprites

The sprite package adds functionality for defining and displaying sprites. To run the demo:

```
include spritedemo
```

Tutorial

Meet the Interpreter

Start up durexForth. The system will greet you with a blinking yellow cursor, waiting for your input. This is the interpreter, which allows you to enter Forth code interactively.

Let us try the traditional first program: Type in `.( Hello, world! )` (and press `Return`). The system will reply `Hello, world! ok`. The `ok` means that the system is healthy and ready to accept more input.

Now, let us try some mathematics. `1 1 +` (followed by `Return`) will add 1 and 1, leaving 2 on the stack. This can be verified by entering `.s` to print the stack contents. Now enter `.` to pop the 2 and print it to screen, followed by another `.s` to verify that the stack is empty.

Let us define a word `bg!` for setting the border color...

```
: bg! $d020 c! ;
```

Now try entering `1 bg!` to change the border color to white. Then, try changing it back again with `0 bg!`.

Introducing the Editor

The `v` editor is convenient for editing larger pieces of code. With it, you keep an entire source file loaded in RAM, and you can recompile and test it easily.

Start the editor by typing `v`. You will enter the red editor screen. To enter text, first press `i` to enter insert mode. This mode allows you to insert text into the buffer. You can see that it's active on the `I` that appears in the lower left corner. This is a good start for creating a program!

Now, enter the following lines...

```
: flash begin 1 $d020 +! again ; flash
```

...and then press `←` to leave insert mode. Press `F7` to compile and run. If everything is entered right, you will see a beautiful color cycle.

When you finished watching, press `RESTORE` to quit your program, then enter `v` to reopen the editor.

Assembler

If you want to color cycle as fast as possible, it is possible to use the durexForth assembler to generate machine code. `code` and `end-code` define a code word, just like `:` and `;` define Forth words. Within a code word, you can use assembler mnemonics.

```
code flash
here ( push current addr )
$d020 inc,
jmp, ( jump to pushed addr )
end-code
flash
```

It is also possible to use inline assembly within regular Forth words:

```
: flash begin [ $d020 inc, ] again ;
flash
```

IMPORTANT

As the x register contains the parameter stack depth, your assembly code must leave it unchanged.

Console I/O Example

This piece of code reads from keyboard and sends back the chars to screen:

```
: foo key emit recurse ;
foo
```

Printer Example

This piece of code prints a message to a printer on device #4, and then prints a message to the screen:

```
include io

: print-hello
4 device ( use device 4 )
0 0 47 7 open ioabort ( open address 7 as file 47, abort on failure )
47 chkout ioabort ( redirect output to file 47, abort on failure )
." Hello, printer!" cr
clrchn ( stop input and output redirection )
." Hello, screen!" cr
47 close ( close file 47 ) ;

print-hello
```

The device number and address may differ between printer models. Commodore MPS series printers use address 0 to print in their uppercase/graphics font, and address 7 to print in their lowercase/uppercase font.

Editor

The editor is a vi clone. Launch it by entering `v foo` in the interpreter (foo being the file you want to edit). You can also enter `v` without argument to create an unnamed buffer. For more info about vi style editing, see [the Vim web site](#).

Inserting Text

At startup, the editor is in command mode. These commands start insert mode, which allows you to enter text. Return to command mode with `↵`.

i

Insert text.

R

Replace text.

a

Append text.

A

Append text at end of line.

C

Change rest of line.

S

Substitute line.

s

Substitute character.

o

Open new line after cursor line.

O

Open new line on cursor line.

cw

Change word.

Navigation

h j k l *or* `←` `↓` `↑` `→`

Move cursor left, down, up, right.

-

Scroll 1 line up.

+

Scroll 1 line down.

Ctrl+u

Half page up.

Ctrl+d

Half page down.

b

Go to previous word.

w

Go to next word.

e

Go to end of word.

fx

Find char x forward.

Fx

Find char x backward.

0 or Home

Go to line start.

\$

Go to line end.

g

Go to start of file.

G

Go to end of file.

H

Go to home window line.

L

Go to last window line.

M

Go to middle window line.

/string

Search forward for the next occurrence of the string.

Search forward for the next occurrence of the word under the cursor.

n

Repeat the latest search.

Saving and Quitting

After quitting, the editor can be re-opened by entering v, and it will resume operations with the edit buffer preserved.

ZZ

Save and exit.

:q

Exit.

:w

Save. (Must be followed by return.)

:w!filename

Save as.

F7

Compile and run editor contents. On completion, enter v to return to editor. To terminate a running program, press **RESTORE**.

Text Manipulation

r

Replace character under cursor.

x

Delete character.

X

Backspace-delete character.

dw

Delete word.

dd

Cut line.

D

Delete rest of line.

yy

Yank (copy) line.

p

Paste line below cursor position.

P

Paste line on cursor position.

J

Join lines.

Forth Words

Stack Manipulation

drop (a —)

Drop top of stack.

dup (a — a a)

Duplicate top of stack.

swap (a b — b a)

Swap top stack elements.

over (a b — a b a)

Make a copy of the second item and push it on top.

rot (a b c — b c a)

Rotate the third item to the top.

-rot (a b c — c a b)

rot rot

2drop (a b —)

Drop two topmost stack elements.

2dup (a b — a b a b)

Duplicate two topmost stack elements.

?dup (a — a a?)

Dup a if a differs from 0.

nip (a b — b)

swap drop

tuck (a b — b a b)

dup -rot

pick ($x_u \dots x_1 x_0$ u — $x_u \dots x_1 x_0 x_u$)

Pick from stack element with depth u to top of stack.

>r (a —)

Move value from top of parameter stack to top of return stack.

r> (— a)

Move value from top of return stack to top of parameter stack.

r@ (— a)

Copy value from top of return stack to top of parameter stack.

depth (— n)

n is the number of single-cell values contained in the data stack before n was placed on the stack.

lsb (— addr)

The top address of the LSB parameter stack.

msb (— addr)

The top address of the MSB parameter stack.

Utility

. (n —)

Print top value of stack as signed number.

u. (u —)

Print top value of stack as unsigned number.

.s

See stack contents.

emit (a —)

Print top value of stack as a PETSCII character. Example: `q emit`

£

Comment to end of line. (Used on C64/PETSCII.)

Comment to end of line. (Used when cross-compiling from PC/ASCII.)

(

Multiline comment. Ignores everything until a `)`. `(` is non-standard: when parsing from an `evaluate` string, it refills to accept multi-line comments.

bl (— char)

char is the PETSCII character for a space.

space

Display one space.

spaces (n —)

Display *n* spaces.

page

Clears the screen.

rvs

Reverse screen output.

Mathematics

 $1+ (a - b)$

Increase top of stack value by 1.

 $1- (a - b)$

Decrease top of stack value by 1.

 $2+ (a - b)$

Increase top of stack value by 2.

 $2* (a - b)$

Multiply top of stack value by 2.

 $2/ (a - b)$

Divide top of stack value by 2.

 $+! (n a -)$

Add n to memory address a.

 $+ (a b - c)$

Add a and b.

 $- (a b - c)$

Subtract b from a.

 $* (a b - c)$

Multiply a with b.

 $/ (a b - q)$

Divide a with b using floored division.

 $/\text{mod} (a b - r q)$

Divide a with b, giving remainder r and quotient q.

 $\text{mod} (a b - r)$

Remainder of a divided by b.

 $*/ (a b c - q)$

Multiply a with b, then divide by c, using a 32-bit intermediary.

****/mod (a b c — r q)***

Like **/*, but also keeping remainder *r*.

0 < (a — b)

Is *a* negative?

negate (a — b)

Negate *a*.

abs (a — b)

Give absolute value of *a*.

min (a b — c)

Give the lesser of *a* and *b*.

max (a b — c)

Give the greater of *a* and *b*.

within (n lo hi — flag)

Return true if $lo \leq n < hi$.

< (n1 n2 — flag)

Is *n1* less than *n2*? (Signed.)

> (n1 n2 — flag)

Is *n1* greater than *n2*? (Signed.)

u < (u1 u2 — flag)

Is *u1* less than *u2*? (Unsigned.)

u > (u1 u2 — flag)

Is *u1* greater than *u2*? (Unsigned.)

lshift (a b — c)

Binary shift *a* left by *b*.

rshift (a b — c)

Binary shift *a* right by *b*.

split (n — lsb msb)

Byte split *n*. \$1234 split gives \$34 \$12.

base (— addr)

addr is the address of a cell that holds the numerical base.

decimal

Set the numerical base to 10.

hex

Set the numerical base to 16.

Double

Double-cell (32-bit) number support is limited. Double literals (e.g. `123456.`) are not supported. This may change with public demand.

The words below are always present. The tests, print and store words are in the `double` module (`include double`), named to match the cell words: where a cell has `0< 0<> 0> 0= < > <> . ! @`, a double has the same ten with a `d` in front. A double sits on the stack as two cells, low half first, so the high half is on top - the order `2@, 2!` and `s>d` already use.

d0< d0<> d0> d0= (d — f)

Compare a double against zero.

d< d> d<> d= (d1 d2 — f)

Compare two doubles. Signed.

d. (d —)

Print a double, signed, followed by a space.

d@ (addr — d)

Fetch a double. Same as `2@`.

d! (d addr —)

Store a double. Same as `2!`.

```
#12345 s>d d.          \ 12345
#30000 s>d #30000 s>d d+ d. \ 60000 - past what a cell holds
#100 s>d #200 s>d d< .   \ -1
```

Unsigned. Three unsigned words are genuinely different from their signed twins; the rest of the pattern is degenerate and deliberately absent. `u0<` would always be false — an unsigned number is never negative — while `u0=`, `u0<>`, `u<>`, `u@` and `u!` are just `0=`, `0<>`, `<>`, `@` and `!`: zero is zero, equality is equality, and bits are bits whichever way you read the sign. The cell versions `u<`, `u>` and `u.` already exist.

du< du> (ud1 ud2 — f)

Compare two doubles, unsigned.

ud. (ud —)

Print a double unsigned, followed by a space.

```
-1 -1 #1 0 d< .   \ -1 : $ffffffff read signed is -1, so it is < 1
-1 -1 #1 0 du< .  \ 0  : read unsigned it is 4294967295, so it is not
-1 -1 d.          \ -1
```

NOTE

d< compares the low halves **unsigned** when the high halves match. It has to: a low half of **\$ffff** means 65535 there, not -1. So **-1 0** is the double 65535, and **-1 0 1 0 d<** is correctly false - a signed low compare would get that backwards.

dabs (d — ud)

Produce the absolute value of *d*.

dnegate (d — d)

Negate the double-cell integer *d*.

s>d (n — d)

Convert the number *n* to the double-cell number *d*.

m+ (d n — d)

Add *n* to double-cell number *d*.

m* (a b — d)

Multiply *a* with *b*, producing a double-cell value.

um* (a b — ud)

Multiply *a* with *b*, giving the unsigned double-cell number *ud*.

um/mod (ud n — r q)

Divide double-cell number *ud* by *n*, giving remainder *r* and quotient *q*. Values are unsigned.

fm/mod (d n — r q)

Divide double-cell number *d* by *n*, giving the floored quotient *q* and the remainder *r*. Values are signed.

Float

The **fp** module (**include fp**) puts floating point on a float stack of its own, as every Forth that takes floats seriously does: the data stack carries addresses and flags, the float stack carries the numbers. So where a cell has **0< 0<> 0> 0= < > <> . ! @**, a float has the same ten with an **f** in front, and none of them disturbs the data stack beyond an address or a flag.

s>f (n —)

Push an integer onto the float stack as a float.

f+ f- f* f/ (—)

Arithmetic on the top two floats.

f0< f0<> f0> f0= (— f)

Compare the top float against zero, dropping it.

f< f> f<> f= (—f)

Compare the top two floats, dropping both.

f. (—)

Print the top float and drop it.

f@ (addr—)

Push the 5-byte float at *addr*.

f! (addr—)

Pop the top float to *addr*.

fdup (—)

Duplicate the top float.

fdrop (—)

Drop it.

fdepth? (—n)

Floats currently on the float stack (8 fit).

```
#7 s>f #2 s>f f/ f. \ 3.5 - real division, not integer
#1 s>f #2 s>f f< . \ -1
create fv 5 allot
#9 s>f fv f! fv f@ f. \ 9
```

NOTE

This is not **float.fs**, which says of itself that it "cannot be used safely" and is right. BASIC's floating point accumulators live at \$61-\$70 and durexForth's parameter stack is \$03-\$72, so every BASIC FP call lands on the data stack; **float.fs** survives only while the stack happens to be shallow. **fp** routes everything through one assembly primitive that parks \$61-\$70, calls the ROM and puts it back, touching the data stack nowhere in between. A float is BASIC's own 5-byte format, so **f.** is FOUT, **f<** is FCOMP and **s>f** is GIVAYF - and BASIC ROM must be present, which the primitive banks in around each call.

Logic

0= (a—flag)

Is *a* equal to zero?

0<> (a—flag)

Is *a* not equal to 0?

= (a b—flag)

Is *a* equal to *b*?

<> (*a b — flag*)

Does *a* differ from *b*?

and (*a b — c*)

Binary and.

or (*a b — c*)

Binary or.

xor (*a b — c*)

Binary exclusive or.

invert (*a — b*)

Flip all bits of *a*.

Memory

! (*value address —*)

Store 16-bit value at address.

@ (*address — value*)

Fetch 16-bit value from address.

c! (*value address —*)

Store 8-bit value at address.

c@ (*address — value*)

Fetch 8-bit value from address.

erase (*addr len —*)

Fill range [addr, len + addr) with 0.

fill (*addr len char —*)

Fill range [addr, len + addr) with char.

move (*src dst len —*)

Copies a region of memory **len** bytes long, starting at **src**, to memory beginning at **dst**.

Compiling

:("*name*" —)

Define the word with the given name and enter compilation state.

:noname (— *xt*)

Create an execution token and enter compilation state.

synonym ("newname oldname" —)

Give an existing word a second name: **synonym** **endif** **then**. The new word behaves exactly like the old — immediacy included, so aliasing compile-time words works — and costs three bytes, its whole body being one **jmp** to the old word. The one thing that cannot reach through a synonym is **to**: it patches the bytes after an execution token, and a synonym's bytes are the jump target, not the value's storage.

; (—)

End the current definition, allow it to be found in the dictionary and go back to interpretation state.

code ("name" —)

Start assembling a new word.

end-code

End assembly.

, (n —)

Write word on stack to **here** position and increase **here** by 2.

c, (n —)

Write byte on stack to **here** position and increase **here** by 1.

allot (n —)

Add *n* bytes to the body of the most recently defined word.

literal (n —)

Compile a value from the stack as a literal value. Typical use: **: x ... [a b *] literal ... ;**

[char] ("c" —)

Compile character *c* as a literal value.

[(—)

Leave compile mode. Execute the following words immediately instead of compiling them.

] (—)

Return to compile mode.

immediate

Mark the most recently defined word as immediate (i.e. inside colon definitions, it will be executed immediately instead of compiled).

['] name (— xt)

Place *name*'s execution token *xt* on the stack. The execution token returned by the compiled phrase **['] x** is the same value returned by **' x** outside of compilation state. Typical use: **: x ... ['] name ... ;**

compile, (xt —)

Append **jsr xt** to the word being compiled. Typical use: `: recurse immmed latest >xt compile, ;`

postpone xxx

Compile the compilation semantics (instead of interpretation semantics) of *xxx*. Typical use:

```
: endif postpone then ; immediate
: x ... if ... endif ... ;
```

header ("name" —)

Create a dictionary header named *name*.

create ("name" —) ... does>

Create a word-creating word named *name* with custom behavior specified after **does>**. For further description, see "Starting Forth."

state (— addr)

addr is the address of a cell containing the compilation-state flag. It is 1 when compiling, otherwise 0.

latest (— value)

Address of the latest defined header.

here (— value)

Write position of the Forth compiler (usually first unused byte of code space). Many C64 assemblers refer to this as program counter or '*'.

marker ("name" —)

Create a word that when called, forgets itself and all words that were defined after it. Example:

```
marker forget
: x ; forget
```

Word List

hide ("name" —)

Remove *name* from the word list, while leaving its definition in place.

define ("name" —)

Assign **here** as the execution token of word *name* and enter the compilation state.

decode ("name" —)

Like **define**, but starts a **code** segment instead.

dowords (*xt* —)

Execute *xt* once for every word in the word list, passing the name token of the word to *xt*, until the word list is exhausted or *xt* returns false. The invoked *xt* has the stack effect ($k * x \ n t \text{---} l * x \ \text{flag}$). If *flag* is true, **dowords** will continue on to the next name, otherwise it will return.

```
\ from debug.fs
: (words) more name>string space 1 ;
: words [''] (words) dowords ;
```

Variables

Values

Values are fast to read, slow to write. Use values for variables that are rarely changed.

1 value *foo*

Create value *foo* and set it to 1.

2 constant *bar*

Create constant value *bar* and set it to 2.

foo

Fetch value of *foo*.

0 to *foo*

Set *foo* to 0.

Variables

Variables are faster to write to than values.

variable *bar*

Define variable *bar*.

bar @

Fetch value of variable *bar*.

1 *bar* !

Set variable *bar* to 1.

Control Flow

Control functions only work in compile mode, not in interpreter.

if ... then

condition IF true-part THEN rest

if ... else ... then

condition IF true-part ELSE false-part THEN rest

do .. loop

Start a loop with index and limit. Example:

```
: print0to7 8 0 do i . loop ;
```

do .. +loop

Start a loop with a custom increment. Example:

```
( prints odd numbers from 1 to n )
: printoddnnumbers (n -- ) 1 do i . 2 +loop ;
```

i, j

Variables to be used inside **do .. loop** constructs. **i** gives inner loop index, **j** gives outer loop index.

leave

Leave the innermost loop.

unloop

Discard the loop-control parameters. Allows clean **((exit))** from within a loop.

```
: x 0 0 do unloop exit loop ;
```

begin .. again

Infinite loop.

begin .. until

BEGIN loop-part condition UNTIL. Loop until condition is true.

begin .. while .. repeat

BEGIN condition WHILE loop-part REPEAT. Repeat loop-part while condition is true.

exit

Exit function. Typical use: **: X test IF EXIT THEN ... ;**

recurse

Jump to the start of the word being compiled.

case .. endcase, of .. endof

Switch statements.

```
: tellno ( n -- )
```

```
case
1 of ." one" endof
2 of ." two" endof
3 of ." three" endof
    ." other" endcase ;
```

Input

key (— *c*)

Get one character from the keyboard.

key? (— *flag*)

Return true if a character is available for **key**.

char (— *c*)

Parse the next word, delimited by a space, and puts its first character on the stack.

>in (— *addr*)

Give the address of a cell containing the offset in characters from the start of the input buffer to the start of the parse area.

refill (— *flag*)

Attempt to fill the input buffer from the input source, returning true if successful.

source (— *caddr u*)

Give the address of, and number of characters in, the input buffer.

source-id (— *n*)

Return 0 if current input is keyboard, -1 if it is a string from **evaluate**, or the current file id.

word (*char* — *addr*)

Read a word from input, using delimiter *char*, and put the string address on the stack. If the delimiter is the space character, non-breaking space (hex a0) will also be treated as a delimiter.

parse (*char* — *addr u*)

Parse a string, using delimiter *char*. *addr* is the address within the input buffer, *u* is the length of the parsed string. If the parse area was empty, the resulting string has a zero length.

parse-name (*name* — *caddr u*)

Read a word from input, delimited by whitespace. Skips leading spaces.

accept (*addr u* — *u*)

Receive a string of at most *u* characters into the buffer that starts at *addr*. Return how many characters were received. This is a deliberately minimal reader — RETURN, DEL and printable characters — used where code asks for a short answer, such as the editor's **:w** filename prompt.

NOTE

The **ok** prompt itself does **not** go through **accept**: the console reads lines through the

Kernal's full-screen editor, the same input BASIC uses. So everything your fingers know from BASIC works at the prompt — CRSR keys, `INST`/`DEL`, editing anywhere on the logical line, `RETURN` taking the line wherever the cursor stands. (Quote mode comes along too: after an unmatched `"`, cursor keys insert reverse-video control glyphs, exactly as in BASIC — press `RETURN` and start over.) If cursor keys type nothing at the prompt under an emulator, the emulator is intercepting them; VICE's "keyset" joystick bound to the arrow keys is the classic culprit. `evaluate ( addr len — ) ::` Evaluate the given string. Evaluate is non-standard: it interprets multi-line strings line-by-line. `quit ::` Enter an endless loop where DurexForth interprets Forth commands from the keyboard. The word is named "quit" since it can be used to quit a program. It also does cleanup tasks like resetting I/O. `pad ( — addr ) ::` `addr` is the address of the `pad`, a 127-byte memory region that can be used freely by user words. No built-in words will modify this region.

Strings

`.(`

Print a string. Example: `.( foo)`

`."`

Compile-time version of `.(`. Example: `: foo ." bar" ;`

`s" ( — caddr u )`

Compilation: Define a string. Interpretation: Parse a string to a transient buffer. Example: `s" foo"`.

`count ( str — caddr u )`

Return data address and length of the counted string `str`.

`type ( caddr u — )`

Print a string.

`/string ( caddr u n — caddr+n u-n )`

Adjust the string by `n` characters.

Cursor position (`include io`):

`at-xy ( x y — )`

Move the cursor to column `x` (0–39), row `y` (0–24); the next `type`, `emit` or `.` prints there. Goes through the Kernal's `PLOT` vector, so the screen-line pointer follows — poking the zero-page cursor bytes directly leaves output on the old line.

`xy@ ( — x y )`

The cursor's current column and row.

```
#10 #5 at-xy ." score:" \ prints at column 10, row 5
```

at-xy output moves the cursor and can scroll the screen like any printing; to write text at a position **without** touching the cursor — mid-game, or from an interrupt — use **text!** from the **charset** module instead (see [Characters and Tiles](#)).

String Manipulation

The **str** module (**include str**) adds friendly words for manipulating strings, which are ordinary (**addr len**) pairs. The slicing words (**left**, **right**, **mid**, **trim**) return a **view** into the same memory — nothing is copied — so a result is valid as long as its source is. The comparison and search words return flags or positions.

left (addr u n — addr' n')

The first *n* characters (fewer if the string is shorter). Example: **s" hello world" 5 left type** prints **hello**.

right (addr u n — addr' n')

The last *n* characters. Example: **s" hello world" 5 right type** prints **world**.

mid (addr u start len — addr' n')

The substring of *len* characters beginning at 0-based offset *start*. Example: **s" hello world" 6 5 mid type** prints **world**.

-leading (addr u — addr' u')

Remove leading spaces.

-trailing (addr u — addr' u')

Remove trailing spaces.

trim (addr u — addr' u')

Remove both leading and trailing spaces.

str= (a1 u1 a2 u2 — f)

True if the two strings are equal.

compare (a1 u1 a2 u2 — n)

Order two strings: **-1** if the first is less, **0** if equal, **1** if greater (byte values; on a common prefix the shorter string sorts first).

index-of (addr u c — i)

The 0-based index of character *c*, or **-1** if not present.

search (a1 u1 a2 u2 — i)

The 0-based index of substring (*a2 u2*) within (*a1 u1*), or **-1** if not present.

starts-with? (a1 u1 a2 u2 — f)

True if the first string begins with the second.

upper (addr u —)

Convert the string to upper case, **in place**.

lower (addr u —)

Convert the string to lower case, **in place**.

>upper (c — C)

Upper-case one character.

>lower (C — c)

Lower-case one character.

cstr (addr u "name" —)

Define *name* as a durable, writable copy of the string. Everything else in the module returns views into the same memory or edits it in place; **cstr** is the one word that copies — take the copy **first** and the original survives whatever you do afterwards. The copy is allotted into the dictionary (u+7 bytes, permanent), and since it defines a word, use it at the top level, not inside a definition.

```
s" hello world" cstr msg
msg cstr backup      \ a copy of the copy
msg upper  msg type   \ HELLO WORLD
backup type           \ hello world - untouched
```

Because **upper** and **lower** modify the bytes in place, give them a writable buffer, not the transient area that **s** returns — a **cstr** copy, or **pad** for scratch work that nothing needs to outlive.

Number Formatting

Pictured numeric output looks impenetrable until you hold two facts, and then it is simple:

1. **The string is built right to left.** **#** produces the **last** digit first, and everything — digits, punctuation — is prepended. That is why the code reads backwards from the output it makes.
2. **It converts a double.** The words work on a 32-bit *ud*, so a single cell needs a **0** on top first (unsigned), or the save-the-sign dance shown below (signed). Doubles arrive on the stack from **s>d**, **um*** and friends — the interpreter itself only accepts 16-bit literals.

<#

Begin the number conversion process.

(ud — ud)

Convert one digit and prepend it to the output string.

#s (ud — ud)

Call **#** repeatedly until *ud* is zero — so at least one digit, and no leading zeros.

hold (ch —)

Prepend the character *ch* to the output string.

sign (n —)

If *n* is negative, prepend a minus sign.

#> (ud — addr u)

End conversion, drop the spent *ud*, and return the string — usually straight into *type*.

Digits obey **base at the moment # runs**, and the string lives in a transient scratch area: *type* it or copy it before converting anything else.

What u. is made of. The whole pattern in one line — 0 makes the double, #s converts every digit:

```
: u. | ( u -- ) 0 <# #s #> type space ;
123 u. | \ 123
```

Leading zeros. Each # is exactly one digit, so six of them are a six-character field — the score counter every game wants:

```
: .score ( u -- ) 0 <# # # # # # #> type ;
1230 .score \ 001230
```

Signed, with punctuation. Building right to left means # # then hold puts the point **before** the last two digits. The sign has to be saved before *abs* and applied after the digits:

```
: .$ ( cents -- ) dup >r abs 0
  <# # # [char] . hold #s r> sign #> type ;
-1234 .$ \ -12.34
```

This is also exactly how *d.* prints a double — see *double.fs*.

Another base, just for this. Conversion uses whatever *base* is when # runs, so switch and restore around it:

```
: .addr ( u -- ) base @ >r hex
  0 <# # # # # [char] $ hold #> type r> base ! ;
49152 .addr \ $c000
```

Two numbers in one picture. The *ud* being converted can be replaced mid-session: convert the seconds' two digits, drop the spent double, push the minutes and keep going. Minutes and seconds, one string:

```
: .mmss ( sec -- ) 60 /mod swap
  0 <# # # [char] : hold 2drop 0 #s #> type ;
```

Right-aligned columns. `#>` hands back the string before anything prints, so measure it and pad first — standard Forth calls this `.r`:

```
: .r ( n width -- ) >r dup >r abs 0 <# #s r> sign #>
  r> over - 0 max spaces type ;
42 8 .r \ 42
```

For the full story, read *Starting Forth*.

Vectored Execution

' ("name" — addr)

Find execution token of the word named *name*.

find (cstr — cstr 0 | xt -1 | xt 1)

Find the definition named in the counted string *cstr*. If the definition is not found, return *cstr* and 0, otherwise return the execution token. If the definition is immediate, also return 1, otherwise also return -1.

find-name (caddr u — 0 | nt)

Get the name token (dictionary pointer) of the word named by *caddr u*, or 0 if the word is not found.

execute (xt —)

Execute the execution token *xt*.

>xt (addr — xt)

Get execution token of word at address *addr*.

Exceptions

throw (n —)

Throw exception *n*, unless *n* is 0. If the exception is not caught, print *n err*, empty the data stack and *quit*.

catch (xt — n)

Execute *xt*, catching any exception. *n* is the exception number, or 0 if nothing was thrown.

Pressing `RESTORE` throws exception -28, "user interrupt". This example shows how to catch it:

```
: idle begin again ;
: demo ." press RESTORE" cr
['] idle catch
```

```
." caught " . ;
```

abort

Throw exception -1. If -1 is not caught, empty the data stack and **quit**.

abort" ccc" (f—)

If *f* is true, throw exception -2. If -2 is not caught, print *ccc*, empty the data stack and **quit**. Typical use: `: x ... test abort" error" ... ;`

Debugging

words

List all defined words.

size ("name" —)

Print the size of the definition of the word named *name*.

dump (n —)

Memory dump starting at address *n*.

n

Continue memory dump where last one stopped.

see ("name" —)

Print the definition of the word named *name*. Works on colon definitions only. Optionally included with **include see**.

Disk I/O

include ("filename" —)

Open and interpret a text file. Example: **include test**

included (filenameptr filenamelen —)

Open and interpret a text file.

require ("filename" —)

Like **include**, except that load is skipped if the file is already loaded.

required (filenameptr filenamelen —)

Like **included**, except that load is skipped if the file is already loaded.

WARNING

Put nothing after a **require on its line — not even a comment.** An **include** reads its file's lines into the 88-byte input buffer at **\$0200**, and when it nests it must first step that buffer past whatever of the outer line has not been read yet. A bare **require** has been read to the end by the time it runs, so it costs nothing and the nested file gets the whole buffer. Add a trailing **\ comment** and

the line is **not** finished, so the buffer steps forward by its full width — and two or three levels of that walks off the end of \$0258 into the Kernal's open-file tables, whereupon the include quietly stops interpreting, or hangs. It fails silently: the words simply are not there afterwards. Keep source lines short for the same reason — a line wider than the buffer is truncated mid-word.

NOTE

Nesting from **disk** is also capped by the drive: a 1541 has three buffers free, so at most three files can be open at once. A four-deep chain answers 70, no channel. Modules that are resident, or served from an EasyFlash cartridge's ROM filesystem, do not consume a drive channel and so do not count. `loadb ( filenameptr filenamelength dst — endaddr ) ::` Load file to *dst*. Returns the address after last written byte, or 0 on failure. `saveb ( start end+1 filenameptr filenamelength — ) ::` Save file. *Start* = start address of memory area. *End+1* = end address of memory area plus 1. `device ( device# — ) ::` Switch the current device. `save-forth ( "filename" — ) ::` Save the forth to the given filename. `ls ::` Load and print disk directory with optional drive # and wildcards. Example: `ls $1:*=p` Load directory for drive 1, only prg files. `rdir ( addr — ) ::` Display disk directory previously loaded to *addr*. `rderr ( — ) ::` Read and print error channel of the current device.

DOS Commands

Words for sending DOS commands to drives and reading drive status are available by including the `dos` module.

`send-cmd ( c-addr u — )`

Write the given string to secondary address 15 on the current device, and print the drive's response. The following example defines a word, `backup` that creates a copy of `durexforth` called `backup`:

```
: backup s" copy0:backup=durexforth" send-cmd ;
backup
```

`dos ( "cmd" — )`

Send *cmd* to the current device's command channel, and print the response. Note that the remainder of the line is treated as part of the command. This makes it possible to refer to file names that contain spaces, but means that `dos` and its command should be on their own line, or the last words on a line. Example: `dos scratch0:old file` will delete a file named *old file*.

The `dos` module also provides a set of shortcuts so you don't have to remember the raw CBM DOS command letters. Like `dos`, each of these takes the remainder of the line as its argument (so file names may contain spaces), and so must be the last word on its line. All operate on drive 0 of the current device (as selected with `device`), which is correct for single-drive units such as the 1541.

`scratch ( "name" — )`

Delete a file. Example: `scratch old file`

rename ("new=old" —)

Rename file *old* to *new*. Example: `rename report=draft`

copy ("new=old" —)

Copy file *old* to a new file *new*. Example: `copy backup=durexforth`

newdisk ("name,id" —)

Format the disk (full low-level format), giving it the name *name* and two-character id *id*.
Example: `newdisk my disk,42`

clear-disk ("name" —)

Quickly clear the disk directory without a low-level format (only valid on an already-formatted disk). Example: `clear-disk my disk`

validate (—)

Validate (collect) the disk: free blocks belonging to improperly-closed files and rebuild the BAM.

initialize (—)

Make the drive re-read the disk's BAM and directory (e.g. after swapping disks).

status (—)

Read and print the drive's status / error channel, e.g. `00, ok,00,00`.

Use the built-in `device` word to choose which unit these operate on, e.g. `9 device`.

Files

The `file` module (`include file`) provides friendly words for reading and writing disk files, both **sequential** (a byte stream read or written start-to-finish) and **relative** (a random-access file of fixed-length records that you can position to by record number). You choose a small file number (2–14) for each open file, so several files can be open at once; 15 is reserved for the command channel.

Sequential files.

fcreate (c-addr u file# —)

Create the named file and open it for writing as *file#*. Example: `s" data" 2 fcreate`

fopen (c-addr u file# —)

Open an existing file for reading.

fappend (c-addr u file# —)

Open a file for appending (writes go after the existing contents).

fputc (c file# —)

Write one byte to *file#*.

fwrite (c-addr u file# —)

Write a string to *file#*.

fgetc (file# — c eof?)

Read one byte from *file#*, and a flag that is true once the last byte has been read. (End-of-file is only known from the read that reaches it, hence the combined result.)

fread-type (file# —)

Read the whole file and print it to the screen.

fclose (file# —)

Close *file#*.

```
include file
: save-it  s" data" 2 fcreate  s" HELLO WORLD" 2 fwrite 2 fclose ;
: show-it  s" data" 2 fopen 2 fread-type 2 fclose ;
```

Relative (random-access) files. A relative file is made of fixed-length records; **record** positions the file to any record, after which the sequential read/write words operate on that record.

rel-create (reclen c-addr u file# —)

Create a relative file with a record length of *reclen* bytes (1–254) and open it as *file#*.

rel-open (c-addr u file# —)

Open an existing relative file.

record (n file# —)

Position *file#* to record *n* (records are numbered from 1). Follow with **fwrite/fputc** to write the record or **fread-type/fgetc** to read it.

rel-close (file# —)

Close a relative file (and its command channel).

```
include file
: make-db  #10 s" people" 2 rel-create  \ 10-byte records
  1 2 record  s" ANN" 2 fwrite
  2 2 record  s" BOB" 2 fwrite
  2 rel-close ;
: read-2    s" people" 2 rel-open
  2 2 record 2 fread-type          \ prints record 2: BOB
  2 rel-close ;
```

Low-Level Device I/O

For more advanced uses, words corresponding to the standard Commodore Kernal IO routines are available by including the **io** module.

open (filenameptr filenamelength file# secondary-addr — ioresult)

Open a logical file.

chkin (file# — ioresult)

Use a logical file as input device.

chkout (file# — ioresult)

Use a logical file as output device.

clrchn (—)

Reset input and output to the keyboard and screen.

close (file# —)

Close a logical file.

readst (— status)

Return the status of the last IO operation. For serial-bus devices, \$01 = write timeout, \$02 = read timeout, \$40 = end of file (EOI), \$80 = device not present.

chrin (— char)

Read a character from the current input device. `ioabort ( ioresult — )` Handle error conditions for `open`, `chkin` and `chkout`. On failure, print error message and throw exception -37.

As per the underlying Kernal routines, `chrin` does not check for end-of-file or any other error condition. `readst` should be called to ensure that the returned character is valid.

The `ioresult` value returned by `open`, `chkin` and `chkout` is 0 on success, or a Kernal error number if an error occurred.

Low-level device I/O may interfere with disk accesses done by `durexForth` and the `v` editor. The following guidelines should be followed to avoid interference:

CAUTION

- Avoid using file numbers 15 and below (remember, any number up to 127 can be used as a file number).
- Only use input/output redirection (`chkin` and `chkout`) within word definitions, and ensure that `clrchn` is called before exit.
- Close files as soon as they are no longer needed.
- If multiple files are open, always call `clrchn` to end any serial bus transactions before calling `open` or switching between files with `chkin` or `chkout`.

Protocol-Level IEC Device I/O

For even more advanced uses, words corresponding to the standard Commodore IEC routines are available by including the `iec` module. These words allow access to serial devices without accessing the file system, and won't hang the computer on a "Device not present" error.

listen (*dv* — *ioresult*)

Send IEC listen to dv.

second (*command*+*sa* — *ioresult*)

Send IEC command and secondary address after listen.

unlisten (—)

Send IEC unlisten to all channels.

ciout (*u* —)

Puts a data byte onto the serial bus using full handshaking.

talk (*dv* — *ioresult*)

Send IEC talk to dv.

tksa (*command*+*sa* — *ioresult*)

Send IEC command and secondary address after talk.

untalk (—)

Send IEC untalk to all channels.

acptr (— *u*)

Get a byte of data from the serial bus using full handshaking.

Commands for **second** and **tksa** include:

- \$60 OPEN CHANNEL / DATA + Secondary Address / channel (0-15)
- \$E0 CLOSE + Secondary Address / channel (0-15)
- \$F0 OPEN + Secondary Address / channel (0-15)

To send a data byte to a drive, that device must first be "listened". If the Secondary address (from here referred to as: SA or channel) is 15, the drive will interpret the data as a DOS command. A DOS command is executed when the drive is UNLISTENed (\$3F). If the channel is not 15, DOS will ignore it unless you first sent an OPEN. An OPEN is sent to tell DOS where you want your data to go. That is done by LISTENing the device.

- channel = 0 is reserved for reading a PRG file.
- channel = 1 is reserved for writing a PRG file.
- channel = 2-14 need the filetype and the read/write flag in the filename as ",P,W" for example.
- channel = 15 for DOS commands or device status info.

After the OPEN is sent, you can send a LISTEN using the channel used in the OPEN. DOS has a table of opened files, and will use the channel to write your data to the corresponding file.

IEC examples

send-cmd (*c-addr u* —)

Demonstrates and documents sending commands and/or reading the error channel.

dos ("*cmd*" —)

Same as Dos command.

bsave (*start end+1 filenameptr filenamelength* —)

Demonstrates and documents sending a file to disk.

dir

Same as `ls`, demonstrates and documents loading a file.

Compatibility

The `compat` module contains various words that are not deemed necessary for enjoyable DurexForth operation, but still must be provided to comply with the Forth 2012 core standard. It also contains some extension words in use by the test suite.

environment? (*addr u* — 0)

Environmental query.

cell+ (*n* — *n+2*)

2+

cells (*n* — *n*2*)

2*

char+ (*n* — *n+1*)

1+

align (—)

No-op

aligned (—)

No-op

chars (—)

No-op

d+ (*d1 d2* — *d3*)

Adds the double-cell numbers *d1* and *d2*, giving the sum *d3*.

2@ (*addr* — *x1 x2*)

Fetch 32-bit value from *addr*. *x2* is stored at *addr*, and *x1* is stored at *addr* + 2.

2! (*x1 x2 addr* —)

Store 32-bit value to *addr*. *x2* is stored at *addr*, and *x1* is stored at *addr* + 2.

2over (*a b c d* — *a b c d a b*)

Copy cell pair *a b* to top of stack.

2swap (*a b c d* — *c d a b*)

Exchange the top two cell pairs.

>number (*ud addr u* — *ud addr2 u2*)

Convert the string in *addr u* to digits, using **base**, and adds each digit into *ud* after multiplying it with **base**. *addr2 u2* contains the part of the string that was not converted.

>body (*xt* — *addr*)

Return the data field address that belongs to the execution token. Example use: ' **foo** >**body**

sm/rem (*d n* — *r q*)

Divide double-cell number *d* by *n*, giving the symmetric quotient *q* and the remainder *r*. Values are signed.

true (— *true*)

Return a *true* flag, a single-cell value with all bits set.

false (— *false*)

Return a *false* flag.

0> (— *n*)

Equivalent to **0 >**.

Kernel Calls

Safe kernel calls may be done from Forth words using **sys (*addr* —)**. The helper variables *ar*, *xr*, *yr* and *sr* can be used to set arguments and get results through the *a*, *x*, *y* and status registers.

Example: '0' **ar c!** **\$ffd2 sys** calls the CHROUT routine, which prints 0 on screen.

Turn-key Utilities

These words are available by including **turnkey**.

top (— *addr*)

Address of the top of the dictionary, default: \$9fff.

top! (*addr* —)

Relocate the dictionary to *addr*. Example:

```
\ not using $a000 block, give all memory to dictionary
```

```
$cbff top!
```

save-pack ("*filename*" —)

Save a compact version of forth to the given *filename*.

save-prg ("*filename*" —)

Save a forth program with no dictionary to *filename*.

Further details on the use of these words are outlined in [Turn-key Operation](#).

Graphics

Turtle Graphics

Turtle graphics are mostly known from LOGO, a 1970s programming language. It enables control of a turtle that can move and turn while holding a pen. The turtle graphics library is loaded with `include turtle`.

init (—)

Initialize turtle graphics.

forward (*px* —)

Move the turtle `px` pixels forward.

back (*px* —)

Move the turtle `px` pixels back.

left (*deg* —)

Rotate the turtle `deg` degrees left.

right (*deg* —)

Rotate the turtle `deg` degrees right.

penup (—)

Pen up (disables drawing).

pendown (—)

Pen down (enables drawing).

turtle@ (— *state*)

Remember turtle state.

turtle! (*state* —)

Restore turtle state as earlier read by `turtle@`.

moveto (*x y deg* —)

Move turtle to `x y` with angle `deg`.

High-Resolution Graphics

The high-resolution graphics library is loaded with `include gfx`. It is inspired by "Step-by-Step Programming Commodore 64: Graphics Book 3." Some demonstrations can be found in `gfxdemo`.

hires (—)

Enter the high-resolution drawing mode.

lores (—)

Switch back to low-resolution text mode.

clrcol (*colors* —)

Clear the high-resolution display using *colors*. *Colors* is a byte value with foreground color in high nibble, background color in low nibble. E.g. `15 clrcol` clears the screen with green background, white foreground.

blkcol (*col row colors* —)

Change colors of the 8x8 block at given position.

plot (*x y* —)

Set the pixel at *x, y*.

peek (*x y* — *p*)

Get the pixel at *x, y*.

line (*x y* —)

Draw a line to *x, y*.

circle (*x y r* —)

Draw a circle with radius *r* around *x, y*.

pen (*mode* —)

Change line drawing method. `1 pen` inverts color, `0 pen` switches back to normal mode.

paint (*x y* —)

Paint the area at *x, y*.

text (*column row str strlen* —)

Draw a text string at the given position. E.g. `10 8 parse-name hallo text` draws the message `hallo` at column 16, row 8.

drawchar (*column row addr* —)

Draw a custom character at given column and row, using the 8 bytes long data starting at *addr*.

Software Sprites (blitter)

The `blit` module (`include blit`) stamps blocks of pixels into the hi-res bitmap.

The VIC gives you eight hardware sprites. Everything else a game draws — a screen full of aliens, a bullet, a piece of landscape — has to be poked into the bitmap by hand, and doing it a pixel at a time with `plot` is far too slow. A blitter moves a whole block at once, optionally combining it with what is already there.

A sprite here is simply *w***h* character cells of 8 bytes each, stored row by row — the same shape `drawchar` takes, only bigger. Build one with `create ... c,.`

blit-blk (src w h x y —)

Stamp the block at cell *x,y*, overwriting.

blit-or (src w h x y —)

OR it into what is there. This is how you draw a shape over a background.

blit-and (src w h x y —)

AND it — a mask.

blit-xor (src w h x y —)

XOR it. Blitting the same data twice returns the bitmap exactly to what it was, which is the cheapest way to move something over a background you cannot afford to save.

blit-get (dst w h x y —)

Read a block back *out* of the bitmap.

blit-col! (c w h x y —)

Colour the cells a blit covers. Hi-res colour is per cell: high nibble is the pixel colour, low nibble the background, so **\$10** is white on black. A sprite blitted without this is there but invisible.

blit-op (— xt)

The word that combines a source byte with the bitmap byte. Assign your own with ' **myop** to **blit-op** and call **blit** if you want something exotic.

Sprite to sprite. These combine one sprite into another of the same size, with the bitmap not involved at all — so no KERNAL banking and no interrupts held off.

cpy-blk cpy-or cpy-and cpy-xor (src dst w h —)

dst becomes **dst op src**.

Build a masked sprite once at startup and a single **blit-blk** per frame then draws it over any background:

```
hole ship cpy-and    \ punch the mask through
art  ship cpy-or     \ lay the artwork in
```

Save the background, draw, then put it back:

```
include blit
: mk #32 0 do $ff c, loop ;
create ship mk          \ a 2x2 cell block, all pixels set
create bg #32 allot
hires $10 clrcl
bg  2 2 #10 #5 blit-get \ remember what was there
ship 2 2 #10 #5 blit-or \ draw over it
bg  2 2 #10 #5 blit-blk \ later: put it back
```

NOTE

Blits are character aligned, as White Lightning's were. The bitmap is stored as 8-byte cells, so a cell-aligned blit is a byte copy; an arbitrary pixel x would mean shifting every byte and merging it across a cell boundary — several times the work. The classic answer is to keep eight pre-shifted copies of the sprite and pick one, which is a job for whatever draws the sprite rather than for the blitter.

NOTE

The bitmap lives at `$e000`, underneath the KERNAL. Writes fall through to the RAM, but reads would come back as KERNAL ROM — and `blit-or`, `blit-and`, `blit-xor` and `blit-get` all read. So every blit is wrapped in `kernal-out/kernal-in`, which means interrupts are off for its duration and nothing may call the KERNAL in between.

Transforming Sprites

The `xform` module (`include xform`) reshapes sprite data. It is cheaper to mirror the alien you already drew than to draw it facing the other way.

Sprites are the `blit` shape — $w \times h$ cells of 8 bytes — so the output feeds straight back into `blit-blk`, or into `sp-data` for a hardware sprite.

`mir ( src dst w h — )`

Mirror left-to-right. `dst` is the same $w \times h$.

`flip ( src dst w h — )`

Flip top-to-bottom. `dst` is the same $w \times h$.

`spin ( src dst w h — )`

Rotate 90° clockwise. Note `dst` comes out $h \times w$ cells — a 1×2 sprite spins into a 2×1 — so size the buffer for that and blit it as `h w`.

`xpandx ( src dst w h — )`

Double the width. `dst` is $2w \times h$ cells.

`xpandy ( src dst w h — )`

Double the height. `dst` is $w \times 2h$ cells.

```
include xform
create ship-l #32 allot
ship ship-l 2 2 mir      \ now blit whichever way it is facing
```

NOTE

These run every pixel through a read and a write, so a 2×2 sprite costs a few hundred thousand cycles. They are meant to run **once**, when your game loads: transform into a second buffer at startup, then blit that buffer every frame.

SID

The `sid` module contains low-level words for controlling the SID chip. To load it, type `include sid`. To test that it works, run `sid-demo`.

Voice Control

voice! (*n* —)

Select SID voice 0-2.

freq! (*n* —)

Write 16-bit frequency.

pulse! (*n* —)

Write 16-bit pulse value.

control! (*n* —)

Write 8-bit control value.

srad! (*srad* —)

Write 16-bit ADSR value. (Bytes are swapped.)

note! (*n* —)

Play note in range [0, 94], where 0 equals C-0. The tuning is correct for PAL.

SID Control

cutoff! (*n* —)

Write 16-bit filter cutoff value.

filter! (*n* —)

Write 8-bit filter value.

volume! (*n* —)

Write 8-bit volume.

Reading the SID

Almost every SID register is write-only. The two exceptions both belong to voice 3, and both are read-only.

osc3@ (— *b*)

Voice 3's oscillator output.

env3@ (— *b*)

Voice 3's envelope output.

`osc3@` is the classic C64 random number source: set voice 3 to a noise waveform and every read gives you a fresh byte. Voice 3 can be muted (bit 7 of `$d418`) so it costs nothing musically, though then you only have two voices for music.

Music

Music Macro Language

Music Macro Language (MML) has been used since the 1970s to sequence music on computer and video game systems. The MML package is loaded with `include mml`. Two demonstration songs can be found in the `mmldemo` package.

MML songs are played using the Forth word `play-mml` which takes three MML strings, one MML melody for each of the three SID voices. An example song is as follows:

```
: frere-jaques
mml" o3l4fgaffgafab->c&c<ab->c&c18cdc<b-l4af>l8cdc<b-l4affcf&ffcf&f"
mml" r1o3l4fgaffgafab->c&c<ab->c&c18cdc<b-l4af>l8cdc<b-l4affcf&ffcf&f"
mml" " play-mml ;
```

Commands

cdefgab

The letters **c** to **b** represent musical notes. Sharp notes are produced by appending a **+**, flat notes are produced by appending a **-**. The length of a note is specified by appending a number representing its length as a fraction of a whole note. For example, **c8** represents a C eight note, and **f+2** an F# half note. Valid note lengths are 1, 2, 3, 4, 6, 8, 16, 24 and 32. Appending a **.** increases the duration of the note by half of its value.

o

Followed by a number, **o** selects the octave the instrument will play in.

r

A rest. The length of the rest is specified in the same manner as the length of a note.

< >

Used to step down or up one octave.

l

Followed by a number, specifies the default length used by notes or rests which do not explicitly specify one.

&

Ties two notes together.

Games and Interrupts

These modules add the pieces needed to write real-time games: attaching a Forth word to the interrupt so it runs in the background, an interrupt-driven music player, and words for joysticks, the mouse, sprite collisions, and VIC-II registers.

Interrupts

The `irq` module (`include irq`) lets you attach a Forth word to the 60 Hz system interrupt. The word then runs in the **background** — 60 times a second — while the foreground interpreter keeps running. This is how games animate sprites, scroll, and play music while still responding to the user.

`irq! (xt —)`

Install `xt` as the interrupt callback. It starts running immediately, ~60 times per second. Calling `irq!` again replaces the callback.

`irq-off (—)`

Stop the callback and restore the previous interrupt handler. Safe to call when nothing is installed.

`?irq (—f)`

True if a callback is currently installed.

`di (—)`

Begin a critical section (disable interrupts). Use around foreground code that touches data the callback also uses.

`ei (—)`

End a critical section (enable interrupts).

Example — flash the border in the background while you keep typing:

```
include irq
: blink 1 $d020 +! ;
' blink irq!      \ border now flickers under interrupt
irq-off           \ stop it
```

The callback contract. The callback runs at interrupt time, so it must be well-behaved:

- It **must not** call the interpreter (`include`, `evaluate`), do disk/IEC I/O, `throw`, or run longer than one frame (1/60 s).
- It **may** read the joystick, poke VIC/SID registers, and update variables.
- It runs on a private data-stack window and its own scratch, so `x`, `a`, `y`, and `w/w2/w3` are saved and restored for you. Keep the foreground data-stack depth below ~42 cells while a callback is installed.

- Guard any variable that both the foreground and the callback modify with `di ... ei`.
- Avoid `do ... loop` inside the callback: it keeps its index on the CPU stack, which clashes with the private data-stack window the callback runs on. Loop with `begin ... until` instead, or do the counting in the foreground.

Raster Interrupts

For effects that must be locked to the beam — split screens, smooth scrolling, tear-free sprite updates — attach the callback to a **raster line** instead of the 60 Hz jiffy. The callback then fires the moment the beam reaches your chosen line.

raster! (*line xt* —)

Install `xt` to fire when the raster beam reaches `line` (0–255). Replaces any previous raster callback. Leaves the Kernal's jiffy interrupt running, so the clock and keyboard keep working.

raster-off (—)

Stop the raster callback, disable the VIC raster interrupt, and restore the previous handler.

?raster (— *f*)

True if a raster callback is currently installed.

The callback contract is the same as for `irq!` (no interpreter/I/O/throw, one frame max, no `do ... loop`, guard shared variables with `di ... ei`).

Example — paint the bottom half of the screen a different colour with a rock-steady split:

```
include irq
include vic
: split 0 border c! 0 bgcol c! ; \ black below line 128
#128 ' split raster!           \ steady horizontal split
raster-off                    \ stop it
```

Because the split is locked to the beam, the boundary never crawls the way a jiffy-timed one would. Only one raster line is armed at a time; for multiple splits, re-latch `vic-raster` (`$d012`) from inside the callback — or use the `split` module, which does exactly that in assembly and saves you the trouble.

Fast callbacks in assembly. For tight, beam-timed work — especially a multi-bar raster that re-arms `$d012` several times per frame — a colon-threaded callback is too slow and will out-run itself. Write the callback in assembly instead. `durexForth` is subroutine-threaded, so a `code` word is called with a plain `jsr` and returns with `rts`, exactly what the trampoline expects; it may freely use `a/x/y` (they are saved for you).

```
include irq
include vic
code flash          \ a native raster callback: a few cycles
  1 lda,# $d020 sta, \ white border below the split
```

```
rts,  
end-code  
#128 ' flash raster!  
raster-off
```

You can also keep the fast bytes inside a normal colon word by dropping into the assembler with [...]; since compilation is subroutine-threaded, those opcodes run natively too. The `rasterbars` example uses a `code` callback to drive eight colour bars per frame — try rewriting it as colon-threaded and it storms the machine, which is the whole reason the callback is assembly.

Split Screen

One screen, several looks: a scrolling playfield above a status panel that holds still, a bitmap half over a text half, a different charset top and bottom. The VIC has one set of registers for the whole frame, so the only way to have two is to change them while the beam is between the bands — which is a raster interrupt, every frame, at every boundary.

The `split` module (`include split`) does that. You describe a band by **setting the screen up the way you want it** and saying where it starts; `band+` photographs the VIC as it stands, so there is no table of magic numbers to get wrong.

bands-clear (—)

Forget every band.

band+ (line —)

Add a band starting at raster *line*, taking the VIC's current registers as its look. Up to 8 bands.

bands-on (—)

Start. `bands-off (—)` :: Stop and hand `$0314` back.

band-sync (—)

Wait for the next frame, locked to the beam.

?bands (—f)

True while bands are running.

Each band carries `$d011`, `$d016`, `$d018`, `$d020` and `$d021` — the two scroll offsets, the graphics mode, the screen and charset pointers, and the two colours. That is everything that makes a band look like itself. A register a band does not care about simply keeps whatever the snapshot caught.

```
include split  
bands-clear  
6 bgcol c! col38          \ the playfield: blue, 38 columns...  
#1 band+                  \ ...from the top of the frame  
0 bgcol c! col40          \ the panel: black, 40 columns...  
#211 band+                \ ...from character row 20 down  
bands-on
```

Raster line for character row N is $51 + 8N$, the display window opening at line 51.

Changing a band while it runs. This is the reason the module exists. $\$d016$ holds the fine x-scroll for the whole frame, so scrolling a playfield with it drags the score panel along too — unless a band puts it back. Set a band's registers with these and the change takes effect on the next frame:

band-xscroll! band-yscroll! ($0..7\ n$ —)

Fine scroll for band n alone.

band-bg! band-border! ($col\ n$ —)

Background/border colour for band n .

band-charset! band-screen! ($addr\ n$ —)

Point band n at its own charset or screen.

```
fine 0 band-xscroll!      \ band 0 glides; every other band is unmoved
```

Where bands can go. Lines 1 to 255, and at least two apart. Line 0 is out because a band is armed one line early, and adjacent lines are out because the raster compare cannot fire twice on the same line — the second band would arrive a frame late.

NOTE

The handler is assembly, and has to be. A colon-threaded callback cannot do this job: by the time the interpreter threaded its way to the register writes the beam would be lines past the boundary, and it could not re-arm $\$d012$ for the next band inside the same frame. All of a band's registers are written in the line **above** it, and that is a considered retreat: the VIC decides at cycle 14 whether a line is a badline (so $\$d011/\$d018$ must arrive early anyway), and on a badline — which every band aligned to a character row starts on — the VIC steals the CPU for cycles 12–54, so a handler that waits for the band's own line to write the colours "invisibly in the border" has them stalled two-thirds of the way across the visible line, at a position that wobbles with interrupt jitter. A wiggling seam, drawn by the fix itself.

The seam rule. Because the registers land a line early, the tail of the line above a band takes the band's colours and scroll. Make that line something that cannot show it — a solid row of reverse spaces in a fixed colour, or the same colours on both sides of the boundary — and the split is pixel-clean. One designed row buys a perfect seam everywhere else; `splitdemo` draws exactly such a black separator row, and keeps $\$d020$ identical across both bands so the side border cannot notch either.

bands-on owns the interrupt. It turns the Kernal's CIA timer interrupt off and makes the VIC raster the only source — because the two fighting is exactly what makes a naïve split flicker. The CIA handler runs some forty rasterlines with interrupts disabled, and whenever that straddles a band's line the raster interrupt is serviced dozens of lines late and the band comes out a whole frame wrong. With the CIA interrupt off, the raster interrupt is serviced within a few cycles every time. The cost is that while bands run there is no jiffy clock and no Kernal keyboard scan: drive your timing from `band-sync`, read keys with `kb?` straight off the matrix, and put per-frame game logic in the foreground after `band-sync` rather than in `irq!` (which the CIA drove, and which is now silent). `bands-off` puts it all back.

band-sync ticks once per frame, released in the lower border just after the last band — which is precisely where a split scroller should do its work. The beam is below the playfield, so a character move there cannot tear, and band 0 does not re-read its scroll until the top of the next frame, so the coarse move and the fine offset always agree. Stepping off the jiffy clock instead — as the first version of the demo did — desynced the two and produced a one-frame jump right, then back left, on every wrap. It also locks the scroll speed to the actual refresh, so the demo runs correctly on both PAL and NTSC.

Example. `include splitdemo` then `splitdemo` — a message gliding across a solid playfield above a score panel that never moves a pixel, both on one screen sharing one scroll register, with no flicker and no tearing on either video standard. RUN/STOP quits.

Background Music

The `mmlirq` module plays **MML** tunes under interrupt, so music keeps going while you do other things. Because `mml` is a compile-time word, wrap the `bg-play` call in a definition, exactly as with `play-mml`.

bg-play (*str1 str2 str3* —)

Start playing three MML voice strings in the background and return immediately.

bg-stop (—)

Stop the music, silence the voices, and uninstall the interrupt.

```
include mmlirq
: song mml" o3l4ccggaagf" mml" o3l4eeff" mml" o3l4cc" bg-play ;
song      \ music now plays in the background
bg-stop   \ stop it
```

Do not run the foreground `play` while background music is active; they share the same SID buffers.

Joystick

The `joy` module reads the joystick ports. Values are already inverted, so a set bit means **pressed**.

joy2 (— *b*)

Read joystick port 2 (`$dc00`). Bits: 0=up, 1=down, 2=left, 3=right, 4=fire.

joy1 (— *b*)

Read joystick port 1 (`$dc01`). Note: port 1 shares the keyboard matrix, so reads are noisy while typing.

joy-up? joy-down? joy-left? joy-right? joy-fire? (*b* — *f*)

Test a direction/fire bit of a value read with `joy1/joy2`.

```
include joy
```

```
: read joy2 joy-left? if ." left " then joy-fire? if ." fire " then ;
```

Mouse

The `mouse` module reads a Commodore 1351 proportional mouse. The 1351 does not report an absolute position — every read gives 6 bits that wrap — so the driver keeps its own `mouse-x` / `mouse-y` and folds in the frame-to-frame movement. Call `mouse-update` once per frame (typically from an `irq!` callback), then read the position and buttons.

`mouse-init (—)`

Prime the driver (read the pots so the first `mouse-update` sees no jump) and centre the pointer in its range. Call after choosing the port and range.

`mouse-update (—)`

Fold this frame's movement into `mouse-x` / `mouse-y`, clamped to the configured range. IRQ-safe (only reads pots and updates variables).

`mouse-x mouse-y (— addr)`

Variables holding the current pointer position.

`mouse-lb? mouse-rb? (— f)`

True while the left / right mouse button is held.

`mouse-port ( n — )`

Select the control port the mouse is in (`1` or `2`). Default is port 1.

`mouse-speed (— addr)`

Variable multiplying each frame's movement (default `1` = 1:1). In VICE the 1351 maps a full window sweep to only ~64 pot units, so the pointer barely travels at 1:1; the `mousedemo` sets `mouse-speed` to `5` so one swipe covers the whole 320-wide screen (and reaches the bottom). On real hardware, `1` or `2` is usually right.

The movement range is set through `mouse-xmin` / `mouse-xmax` / `mouse-ymin` / `mouse-ymax` (default `0..319` by `0..199`); set them to suit sprite coordinates, then `mouse-init`. These bounds are the only thing that limits how far the pointer travels — if it seems to stop short of an edge, widen them (it is not a hardware or 9th-bit limitation). Sprite X is a 9-bit coordinate (0–511) and positions past 255 set the sprite's 9th X bit automatically via `sp-x!`; sprite Y is a single byte. Because the sprite's position is its top-left corner and the visible screen is roughly x 24–320, y 50–250, `mousedemo` clamps to `24..343` by `50..249` — tuned on real hardware so the arrow's tip reaches every edge while at least one pixel of the sprite stays on screen. That makes the pointer usable as a paint cursor that can address the whole visible canvas.

```
include mouse
mouse-init
: track mouse-update mouse-x @ mouse-y @ 0 sp-xy! ;
' track irq!           \ sprite 0 now follows the mouse
```

The `mousedemo` module is a complete example: an arrow sprite follows the mouse and turns red while the left button is held (`mouse-demo` / `stop-mouse-demo`). In VICE, enable a 1351 mouse on port 1 and grab it. Because the buttons come through the joystick lines, port-1 button reads are noisy while typing, exactly as with `joy1`.

Sprite Helpers

The `sprite` module adds words for multicolor sprites, priority, data pointers, and hardware collision detection (on top of the position/enable/colour words).

sp-mc-on sp-mc-off (*n* —)

Turn multicolor mode on/off for sprite *n*.

sp-mc0! sp-mc1! (*c* —)

Set the two shared sprite multicolor registers.

sp-behind sp-front (*n* —)

Put sprite *n* behind / in front of the background.

sp-ptr! (*block n* —)

Set sprite *n*'s data pointer to a 64-byte *block* number. (Sprite data at address *A* uses block *A/64*.)

sp-sp-coll (— *b*)

Read the sprite-sprite collision register (`$d01e`). Cleared on read, so read once per frame.

sp-bg-coll (— *b*)

Read the sprite-background collision register (`$d01f`).

sp-hit? (*b n* — *f*)

Test whether sprite *n* collided, given a byte read from `sp-sp-coll/sp-bg-coll`.

sp-x@ sp-y@ (*n* — *x*) / (*n* — *y*)

Read sprite *n*'s position back. `sp-x@` reassembles the 9-bit *x* from `$d000+2n` and the overflow bit in `$d010`.

The collision registers only tell you *which* sprite hit something, so reading the position back is how you act on it:

```
sp-sp-coll 3 sp-hit? if
  3 sp-x@ 3 sp-y@ explode
then
```

Keyboard

The `keyb` module (`include keyb`) reads the keyboard matrix directly.

`key` and `key?` go through the KERNAL, which is the wrong shape for a game: they are buffered, they

deliver one keypress at a time, and they repeat. A game needs to know what is held down **right now**, and needs several keys at once — "left and fire" is one state, not two events, and no amount of **key?** will tell you that.

kb? (n—f)

True if key *n* is held down at this instant.

kb-key (row col—n)

Build a key number from a matrix position.

The matrix is 8 rows by 8 columns, so a key is just **row*8 + col** — 0 to 63. Named constants cover the keys games actually use: **k-space**, **k-ctrl**, **k-lshift**, **k-rshift**, **k-return**, **k-stop**, **k-a**, **k-s**, **k-d**, **k-w**, **k-z**, **k-x**, **k-c**, **k-q**, **k-crsr-dn**, **k-crsr-rt**, **k-f1**, **k-f3**, **k-f5**, **k-f7**. For anything else use **kb-key** with a matrix table from any C64 reference.

```
include keyb
: fly
  k-z kb? if -1 move-left then
  k-x kb? if 1 move-right then
  k-space kb? if fire then ; \ all three at once, if you like
```

The **keybdemo** module is a hands-on check, since an emulator harness cannot press a real key — **-keybuf** fills the KERNAL's buffer and never touches the matrix:

```
include keybdemo
keys \ hold keys - several at once; RUN/STOP quits
```

It names the keys it knows and shows the raw matrix number of anything held, so you can identify a key that has no constant.

NOTE

There are only **four** function keys in the matrix. F2 is SHIFT+F1, F4 is SHIFT+F3, F6 is SHIFT+F5, F8 is SHIFT+F7 - so pressing F2 makes **kb?** report both **k-f1** and **k-lshift**. That is the hardware being honest, not a bug. To tell them apart:

```
: f1? k-f1 kb? k-lshift kb? 0= and ;
: f2? k-f1 kb? k-lshift kb? and ;
```

NOTE

The two shift keys are separate keys in the matrix (**k-lshift** = 15, **k-rshift** = 52), not one. Check both if you mean "either shift".

NOTE

kb? scans with interrupts held off for a few cycles, because the KERNAL scans the same hardware sixty times a second and would otherwise move the row out from under the read. It uses **php/plp** rather than **sei/cli**, so it is safe to call from inside an **irq!** callback. It also leaves **\$dc00** at **\$ff** on the way out — with a row still selected, **joy2** reads the same port and would report phantom directions.

Characters and Tiles

The `charset` module (`include charset`) covers the other way to draw a game. The C64 is a character machine: a screen of characters is 1000 bytes instead of the bitmap's 8000, it redraws in a fraction of the time, and you get colour per cell for free. Most C64 games build their world out of a redefined 8x8 font rather than the bitmap.

Pointing the VIC. The VIC can only see one 16K bank at a time, and a screen or charset must live inside it. Colour memory is the exception — it is wired at `$d800` regardless.

vic-bank@ (— *n*)

The bank the VIC is currently looking at, 0-3. (Bank 0 is `$0000`, bank 3 is `$c000`.)

vic-bank! (*n* —)

Point the VIC at bank *n*.

vic-base (— *addr*)

Base address of the current bank.

screen@ screen! (— *addr*) / (*addr* —)

Get/set the screen memory address. Must be `$400`-aligned and inside the current bank. `screen!` also moves `sp-base` with it, since the sprite data pointers live at `screen+$3f8`.

charset@ charset! (— *addr*) / (*addr* —)

Get/set the character set address. Must be `$800`-aligned and inside the current bank.

charset-lock (—)

Stop the KERNAL swapping your font out. Call it before `charset!`.

charset-unlock (—)

Allow the swap again.

WARNING

The KERNAL switches between the two ROM fonts when SHIFT+C= is pressed, and some hosts (MiSTer among them) reach the same toggle from CAPS. It does it by flipping `$d018` - the register `charset!` just set - so with a custom charset this is not a cosmetic annoyance: your font is replaced by a ROM one mid-game and the screen turns to garbage. `charset-lock` (`$0291` bit 7) turns the toggle off. Any game with its own font wants it.

NOTE

In practice your charset cannot live in bank 0. `durexForth`'s own code runs from `$0801` up to `here` (around `$38xx`), which swallows every `$800`-aligned slot in `$0000-$3fff`. So a custom charset means moving the VIC to bank 1 (`$4000`), where there is free RAM between `here` and the dictionary — and the screen has to move with it, because the VIC sees one bank at a time.

NOTE

`durexForth` prints through the KERNAL, which always writes to `$0400`. Once the screen has moved, the prompt goes on writing to `$0400` and you stop seeing what

you type. That is expected — a game draws its own screen — but it means that to **watch** a font change you have to bring the current text along, as the example does.

The ROM font. The character generator ROM is not normally reachable: it only appears, over the I/O area at `$d000`, when `CHAREN` is 0. `charrom>` handles the banking (and the interrupt lock it needs, since the KERNAL's IRQ cannot ack the timer while I/O is hidden), so you can take a copy and edit it.

charrom (— *addr*)

Address of the upper case + graphics set in ROM.

charrom-lc (— *addr*)

Address of the lower case set. This is the one `durexForth` itself displays.

charrom> (*src dst* —)

Copy one 2K set out of ROM to RAM.

chardef (*c* — *addr*)

Address of character *c*'s 8 bytes in the current charset.

char-data (*addr* —)

Read 8 rows of pixels from the source, like `sp-data`: any character that is not `.` sets a pixel.

Take a copy of the font into bank 1, bring the text along so there is something to look at, point the VIC at it, then redefine a character — every `a` on screen changes at once:

```
include charset
charrom-lc $4800 charrom> \ copy the font we are looking at
$400 $4000 #1000 move \ carry the current text over to the new screen
1 vic-bank! $4000 screen! $4800 charset!
1 chardef char-data \ screen code 1 is "a"
#####
#.....#
#.....#
#####
#.....#
#.....#
#.....#
#.....#
#####
```

Tiles. A tile is nothing more than *w***h* character codes stored row by row, so a 2x2 tile is four bytes. Build them with `create ... c,.`

scr-at (*x y* — *addr*)

Screen address of character cell *x,y*.

col-at (*x y* — *addr*)

Colour memory address of cell *x,y*.

tile! (addr w h x y —)

Stamp a *w* by *h* tile onto the screen at *x,y*.

tile-col! (col w h x y —)

Paint colour *col* over the same *w* by *h* area.

```
create brick 1 c, 2 c, 3 c, 4 c,  
brick 2 2 #10 #5 tile!      \ stamp it at column 10, row 5  
2 2 2 #10 #5 tile-col!     \ and colour it red
```

Text on the playfield. Screen memory holds *screen codes*, not PETSCII — poke a string from *s* in raw and every letter is off by an alphabet. **text!** converts and writes in one go, straight into screen memory: no cursor, no scrolling, no Kernal, so the prompt stays where it was and it is safe from an **irq!** callback. (For ordinary cursor-and-scroll output, **at-xy** and **type** — see [the I/O words](#).)

text! (addr u x y —)

Write the string at character cell *x,y*.

p>s (c — c')

Convert one PETSCII character to its screen code — both letter cases, digits and punctuation; control codes pass through unchanged.

```
s" score" 0 #24 text!  
1 5 1 0 #24 tile-col!    \ white, same five cells
```

Scrolling

The **scroll** module (**include scroll**) is the other half of tile-based games: **tile!** builds a world, scrolling moves the camera over it.

Scrolling a C64 screen is two jobs stitched together. The VIC will shift the whole display 0-7 pixels for free — that is the smooth part, and it costs one register write. But 7 pixels is all it can do, so every 8th pixel you move the characters themselves one cell, reset the fine offset back where it started, and draw the newly exposed edge. Done right the two are invisible and the picture simply keeps moving.

Fine (hardware) scroll.

xscroll@ xscroll! (— 0..7) / (0..7 —)

Get/set the horizontal fine offset (**\$d016** bits 0-2).

yscroll@ yscroll! (— 0..7) / (0..7 —)

Get/set the vertical fine offset (**\$d011** bits 0-2). Note the power-on value is 3, not 0.

col38 col40 (—)

Narrow the display to 38 columns, or back to 40.

row24 row25 (—)

Narrow the display to 24 rows, or back to 25.

TIP

A side-scroller wants `col38`. Without it the column being scrolled in is drawn half-finished at the screen edge, and it flickers. Narrowing makes the VIC cover that edge with border. `row24` does the same for a vertical scroller.

Character scroll. These shift a w by h window of the screen one whole cell. The vacated edge is left as it was rather than cleared — the caller draws the new column or row into it, which is the entire point of scrolling.

scroll-left scroll-right (x y w h —)

Shift the window one character sideways.

scroll-up scroll-down (x y w h —)

Shift the window one character vertically.

col-scroll (—f)

Whether colour memory is shifted too. Default is on.

Wrapping. The plain `scroll-*` leave the vacated edge alone, because a scroller is about to draw new material into it. When you want the window's contents to cycle forever instead — a marquee, a repeating background — use the wrapping versions: whatever leaves one edge comes back at the other, and nothing has to feed them.

wrap-left wrap-right (x y w h —)

Shift one character sideways, carrying the lost column round to the far edge.

wrap-up wrap-down (x y w h —)

The same vertically.

The `scrolldemo` module is built out of these — see below.

Colour is shifted along with the characters, because colour memory does **not** move with the screen — it is wired at `$d800` whatever `screen!` says. That doubles the work, so a game with a uniform colour scheme can turn it off:

```
0 to col-scroll
```

Putting both halves together — this is the whole of a side-scroller's per-frame work:

```
include scroll
0 value fine
: bump fine 1- to fine ;
: wrapf 7 to fine 0 #15 #40 1 scroll-left ;
: step bump fine 0< if wrapf then fine xscroll! ;
```

`step` moves row 15 one pixel left. Every eighth call `fine` runs past zero, so the row is shifted one whole character and the fine offset snaps back to 7 — leaving the picture exactly where the eye expects it.

WARNING

Drive that from the foreground, one step a frame — **not** from `irq!`. `scroll-left` and friends use `do ... loop`, which is illegal in a callback: it `tsx'es` onto the CPU stack and collides with the private data-stack window the callback runs on, and surfaces as a stack underflow in the foreground. Syncing to the jiffy clock (`$a2``) gives exactly one step per frame and costs nothing.

Example: banners. The `scrolldemo` module has two, one per direction — run them one at a time, which is the point. With a single banner on screen the VIC's fine scroll is free to move it a pixel at a time and the result glides; two banners would have to share that one register and neither could have it.

```
include scrolldemo
hbanner          \ a sentence gliding left to right
vbanner          \ a word gliding top to bottom
```

RUN/STOP stops either. They wrap rather than scroll, so the sentence cycles forever with nothing feeding it new characters, and they hold the screen rather than returning to the prompt — `ok` and a blinking cursor would land right on top of the banner. `bspeed` is frames per pixel: 8 is about a character a second, which is a banner you can read.

Eight-Way Scrolling

`scroll.fs` shifts what is already on the screen, one edge at a time, and leaves you to draw the newly exposed strip. That is right for a banner and wrong for a world: a game does not scroll the screen, it moves a **camera** over a map, and the screen shows whatever the camera is looking at.

The `scroll8` module (`include scroll8`) does that. Say where the camera is and the screen follows — which direction it moved, or whether it moved diagonally, stops being something anyone has to think about. A map is simply w by h bytes of character codes, so `CharPad's` exported map drops straight in.

map! (*addr w h* —)

Use the w by h map at *addr*.

map-col! (*addr* —)

Its colour map, same shape. Leave it unset for a one-colour world — see the cost below.

win! (*x y w h* —)

The window on screen the camera draws into.

cam! (*x y* —)

Put the camera at pixel x,y in the map.

cam+ (dx dy —)

Move it, clamped so it cannot walk off the map.

cam-x cam-y (— n)

Where it is, in pixels.

cam-start (—)

Turn on col38/row24 and draw the first frame.

cam-stop (—)

Put the display back to 40x25.

```
include scroll8
the-map #64 #48 map!
the-col map-col!
0 0 #40 #25 win!
cam-start
1 1 cam+           \ one pixel right and down: diagonal is free
cam-stop
```

Why the arithmetic looks odd. The VIC can shift the display **right** by 0–7 pixels and **down** by 0–7, and that is all — there is no shift left and no shift up. So to move the world left by one pixel you step the character window one cell on (8 pixels left) and shift 7 back to the right. Net: one pixel. Both axes are the same rule around a different neutral — x rests at 0, y at 3, because `$d011`'s rest value is ``$1b`:

```
cell    = (cam + 7 - neutral) >> 3
scroll = (neutral - cam) & 7
```

`cam-start` turns on `col38` and `row24`, which is why the screen goes a character narrower: when the offset is mid-cell the strip coming in at the edge is only part drawn, and narrowing makes the VIC paint border over exactly that strip.

What it costs. The window is redrawn from the map only when the camera crosses a cell boundary, so 7 frames in 8 are one register write and free. The 8th copies `win-h` rows, twice over if the map has colour. Measured on the cartridge:

window	per redraw	PAL frames
40x25 with colour	43 ms	~2
40x25 no colour	19 ms	~1
40x16 no colour	11 ms	~0.5

So the colour map is not a detail — it is **half the cost**, and dropping it is the single biggest lever you have. At 40x25 in colour the redraw overruns its frame and the scroll pauses for two every eighth pixel; in one colour it just fits; a shorter window fits with room to spare. Scrolling a pixel every

other frame makes even the worst case read as smooth, because the stall is a smaller share of the time between steps.

NOTE

`cam!` redraws with `do ... loop`, so like the rest of `scroll`, drive it from the foreground rather than a callback.

Example. `include scroll8demo` then `scroll8demo` — a camera over a 64x48 map, moving a pixel a frame in whatever direction you push the joystick in port 2, diagonals included. Each 8x8 block is labelled with its own column and row digits, so you can read straight off the screen whether the camera is where it claims; hold a diagonal into a corner and it clamps cleanly instead of scrolling off the end of the map. `RUN/STOP` quits.

VIC-II Register Constants

The `vic` module (`include vic`) defines readable names for the VIC-II registers — `border`, `bgcol`, `vic-ctrl1`, `vic-raster`, `sp-enable`, `sp-sp-hit`, and so on — so game code reads clearly instead of using bare addresses. See the module source for the full list.

Example: a bouncing ball

The `bounce` module is a complete example. It defines a ball sprite and animates it around the screen under interrupt, so the ball keeps bouncing while you type at the `ok` prompt.

```
include bounce
bounce          \ the ball starts bouncing; you get the prompt back
stop-bounce     \ stop it and hide the sprite
```

Example: raster colour bars

The `rasterbars` module shows `raster!` driving a classic effect: eight horizontal colour bars, each locked to its own raster line, painted while you keep typing at the prompt. Its callback is an assembly `code` word — it re-arms `$d012` once per bar, eight times a frame, which only native code is fast enough to do without out-running the beam.

```
include rasterbars
bars            \ colour bars appear; the prompt returns
stop-bars      \ stop them and restore a plain screen
```

Importing Graphics and Music

You will not draw a game's graphics or write its music at the durexForth prompt, and you should not try. The C64 has good tools for that — SpritePad Pro and CharPad Pro for graphics, GoatTracker 2 for music — and durexForth can load what they produce.

The trick is that you never parse their project files. A `.spd` or `.ctm` is an editor document, full of undo history and palette state. Every one of these tools exports plain binary as well, and that is what a C64 wants: bytes, at an address.

The catch that bites everyone: where it goes

durexForth is a large program and it is **in the way**. On the cartridge build:

<code>\$0801 - \$4a9a</code>	durexForth's code, up to <code>here</code>
<code>\$4a9b - \$8c6d</code>	free — about 16K, put your assets here
<code>\$8c6e - \$9cff</code>	the dictionary, growing down
<code>\$9d00 - \$9f27</code>	the cartridge's romfs driver

Character data and sprite data are just bytes, so they go anywhere free that is correctly aligned. **Music does not.** A SID player has its addresses baked into its own code — it cannot be loaded somewhere other than where it was built. GoatTracker defaults to `$1000`, which lands squarely on top of the interpreter and takes it out. Build or pack the tune for about `$5000` instead.

The other catch: loadb eats two bytes

`loadb ( addr len dst—endaddr )` asks the KERNAL to load with a secondary address of 0, which means the file's first two bytes are treated as a load address and thrown away. That is correct for a `.prg`, and wrong for a raw `.bin` — you would silently lose the first two bytes of your charset.

So give it a `.prg`. Either export one, or glue a two-byte dummy address on the front (the same guard durexForth's own modules use):

```
printf '\x00\x48' | cat - charset.bin > chars.prg
c1541 -attach game.d64 -write chars.prg chars
```

CharPad Pro

Export **charset** (raw, 2048 bytes = 256 characters) and, if you use per-character colour, **attributes** (256 bytes, one per character). Then it is two words:

```
include charset
s" chars" $4800 loadb drop    \ 2K -> $4800-$4fff
$400 $4000 #1000 move        \ bring the text along to look at
```

```
1 vic-bank! $4000 screen! $4800 charset!
```

The charset must be **\$800**-aligned and inside the VIC bank, and the screen has to move into that bank with it — see [Characters and Tiles](#). Export the **map** as raw binary too and draw it with **tile!**.

SpritePad Pro

Export **sprites** as raw binary: 64 bytes each, back to back, so 80 sprites is 5120 bytes. Load it 64-byte aligned and point a sprite at it. The pointer is a block number counted from the VIC bank, not an address — $\text{block} = (\text{address} - \text{bank base}) / 64$:

```
include charset
s" sprites" $5000 loadb drop \ 80 sprites at $5000
1 vic-bank! $4000 screen!
#64 0 sp-ptr! \ ($5000-$4000)/64 = 64 -> sprite 0
#100 #100 0 sp-xy! 1 0 sp-col! 0 sp-on
```

GoatTracker 2

A **.sid** file is a 124-byte PSID header followed by the C64 payload — and that payload already begins with its own two-byte load address, so with the header off it is a **.prg** and **loadb** takes it unchanged:

```
dd if=tune.sid bs=1 skip=124 of=tune.prg
```

(124 is **dataOffset**, the big-endian word at offset 6 of the header; it is **\$7c** for PSID v2. GoatTracker's own packer can emit the **.prg** directly and save you the step.)

The payload is a player with two entry points — almost always **jmp init** at the load address and **jmp play** three bytes later. The **sidmusic** module wires them up:

sid-load (*addr len dst* —)

Load a tune and assume init at **dst**, play at **dst+3**.

sid! (*init play* —)

Point at the two entry points explicitly, for a tune that is laid out differently.

sid-init (*song* —)

Call the tune's init with a song number.

sid-on (—)

Start calling play every frame.

sid-off (—)

Stop, and silence the SID.

```
include sidmusic
s" tune" $5000 sid-load
0 sid-init
sid-on          \ music plays while you carry on working
sid-off
```

NOTE `sid-on` installs a native `jmp play` for the interrupt to `jsr` into. It has to be machine code: a colon-threaded callback is too slow for beam-timed work, and anything using `do ... loop` is illegal in a callback outright.

NOTE `sid-on` uses `irq!`, which replaces any other callback. To run music **and** your own per-frame word, call both from one word — Forth composes, so you do not need anything cleverer:

```
: frame sid-tick move-sprites ;
```

...where `sid-tick` is your own `code` word doing `jsr $5003`, or simply let `sid-on` own the interrupt and drive your game from the foreground.

EasyFlash Cartridge

durexForth can be built as an EasyFlash cartridge: `deploy/durexforth-ef.crt`. It boots straight to a durexForth prompt with the game-dev modules already resident, and it carries every module in flash, so `include` works with **no disk attached**. This is the build to use on a MiSTer or an EasyFlash 3.

What is resident

The cartridge boots with these modules already compiled in — no `include` needed:

`vic, irq, joy, sprite, mouse, sid, mml, mmlirq, rnd.`

That means `irq!`, `raster!`, `joy1`, `sprite` and `SID` words all work the moment you get a prompt. This is the whole point of the cartridge: the packed image is ~17K, which does not fit the 16K of a plain cartridge, so it is split over two EasyFlash banks.

The editor `v` is **not** resident — it is large, and you do not need it in every game. `include v` loads it from flash in a moment. `timer` is not resident either, for a specific reason: it defines a word called `start`, which would shadow the kernel's own `start` and break the packing step. `include timer` after boot is perfectly fine and gives you the normal `start ... stop`.

Including from flash

Everything else — `v`, `dos`, `file`, `str`, `timer`, `see`, `gfx`, the demos — lives in the cartridge's flash, so `include` just works:

```
include v          \ the editor, straight from flash
include fractals    \ pulls in turtle too, also from flash
```

There is no disk involved, and nothing to mount. If you **do** have a drive attached, it still works: names that are not in flash fall through to the real KERNAL, so `include mygame` loads from disk 8 as usual. Flash is searched first.

The cartridge holds 42 modules in about 111K of its 1 MB flash, leaving plenty spare.

Memory

The cartridge keeps a small driver resident in RAM to serve `include` from flash. It hooks the KERNAL's `OPEN/CHKIN/CHRIN/CLOSE` vectors, so anything that reads a file — `include`, the editor's `:e`, `dos` — transparently sees flash files.

romfs driver The driver occupies `$9d00-$9f27`, and on this build the dictionary therefore ends at `$9cff` instead of `$9fff`. You lose 768 bytes of dictionary space; everything else is unchanged. This is the only hole available: `$a000-$cbff` is the editor text buffer and `$cc00-$cfff` is hi-res colour, so a driver parked up there would be destroyed the first time you edited a file or went hi-res.

If you are writing a program that stomps `$9d00-$9f27`, `include` will stop working (and probably crash). Everything below `$9cff` and `$a000` upward behaves exactly as the appendix memory map describes.

Building the cartridge

The cartridge is not built by `make`; the steps are documented at the top of `asm/efbuild.py`, which lays out the flash image. In outline: assemble the kernel, `petcat` the modules, boot a packing disk once so `base-ef.fs` packs itself, then:

```
acme --symlist build/cart-ef.sym asm/cart-ef.asm
python asm/efbuild.py
cartconv -t easy -i deploy/durexforth-ef.bin \
        -o deploy/durexforth-ef.crt -n "DUREXFORTH EF"
```

`forth/base-ef.fs` is the cartridge's `base` — it chooses the resident set. `asm/cart-ef.asm` is the boot stub and the flash filesystem driver.

TIP

When testing the cartridge in VICE, pass `+easyflashcrtwrite`. VICE writes the emulated flash back over your `.crt` on exit by default, silently rewriting the file you just built.

Assembler

DurexForth features a simple but useful 6510 assembler with support for branches and labels. Assembly code is typically used within a **code** word, as in the tutorial example:

```
code flash
here      ( push current addr )
$d020 inc, ( inc $d020 )
jmp,      ( jump to pushed addr )
end-code
```

It is also possible to inline assembly code into a regular Forth word, as seen in the tutorial:

```
: flash begin [ $d020 inc, ] again ;
```

Variables

DurexForth has a few variables that are specifically meant to be used within code words.

lsb (— *addr*)

addr points to the top of the LSB parameter stack. Together with the **x** register, it can be used to access stack contents.

msb (— *addr*)

addr points to the top of the MSB parameter stack. Together with the **x** register, it can be used to access stack contents.

w (— *addr*)

A zero-page cell that code words may use freely as work area.

w2 (— *addr*)

Second zero-page work area cell.

w3 (— *addr*)

Third zero-page work area cell.

Example usage of **lsb** and **msb**:

```
code + ( n1 n2 -- sum )
clc,      ( clear carry )
lsb 1+ lda,x ( load n1 lsb )
lsb adc,x   ( add n2 lsb )
lsb 1+ sta,x ( store to n1 lsb )
msb 1+ lda,x ( load n1 msb )
msb adc,x   ( add n2 msb )
msb 1+ sta,x ( store to n2 msb )
```

```
inx,          ( drop n2; n1 = sum )
rts,          ( return )
end-code
```

Branches

The assembler supports forward and backward branches. These branches cannot overlap each other, so their usage is limited to simple cases.

+branch (— *addr*)

Forward branch.

:+ (*addr* —)

Forward branch target.

:- (— *addr*)

Backward branch target.

-branch (*addr* —)

Backward branch.

Example of a forward branch:

```
foo lda,
+branch beq,
bar inc, :+
```

Example of a backward branch:

```
:- $d014 lda, f4 cmp,#
-branch bne,
```

Labels

The **labels** module adds support for more complicated flows where branches can overlap freely. These branches are resolved by the **end-code** word, so it is not possible to branch past it.

@: (*n* —)

Create the assembly label *n*, where *n* is a number in range [0, 255].

@@ (*n* —)

Compile a branch to the label *n*.

Example:

```
code checkers
$7f lda,# 0 ldy,# 'l' @:
$400 sta,y $500 sta,y
$600 sta,y $700 sta,y
dey, 'l' @@ bne, rts,
end-code
```

Configuring durexForth

Stripping Modules

By default, durexForth boots up with these modules pre-compiled in RAM:

asm

The assembler. (Required and may not be stripped.)

format

Numerical formatting words. (Also required.)

wordlist

Wordlist manipulation. (Required by some modules.)

labels

Assembler labels.

doloop

Do-loop words.

sys

System calls.

debug

Words for debugging.

ls

List disk contents.

require

The words `require` and `required`.

v

The text editor.

To reduce RAM usage, you may make a stripped-down version of durexForth. Do this by following these steps:

1. Issue `---modules---` to unload all modules, or `---editor---` to unload the editor only.
2. One by one, load the modules you want included with your new Forth. (E.g. `include labels`)
3. Save the new system with e.g. `save-forth acmeforth`.

Custom Start-Up

You may launch a word automatically at start-up by setting the variable `start` to the execution

token of the word. Example: ' **megademo start** ! To save the new configuration to disk, type e.g. **save-forth megademo**.

When writing a new program using a PC text editor, it is practical to configure durexForth to compile and execute the program at startup. That can be set up using the following snippet:

```
$a000 value buf
: go buf s" myprogramfile" buf
loadb buf - evaluate ;
' go start !
save-forth @0:durexforth
```

Turn-key Operation

Durexforth offers utilities to save your program in a turn-key fashion by including the **turnkey** module once the program is ready to be saved.

Programs can be saved in a compacted state using **save-pack**. These programs are stored with 32 bytes between **here** and **latest**. When they are first loaded, they will restore the header space to its original **top**.

If you have developed a program that has no further need of the interpreter, you can eliminate the dictionary headers entirely when saving with **save-prg**. This allows your program to use memory down to **here** plus 32 bytes for safety.

After either of these words have saved the file to disk, they will restore forth to the unpacked state, and strip the **turnkey** module from the dictionary. This allows you to continue to use forth interactively in the case of **save-pack**. As **save-prg** has stripped the dictionary headers from the system, it will no longer be usable. If you wish to test your program after saving, you can compile a call to **save-prg** instead:

```
: build save-prg mydemo start @ execute ;
build
```

This will simulate the start-up sequence after saving the packed program.

Appendix A: Assembler Mnemonics

adc,#	adc,	adc,x	adc,y	adc,(x)	adc,(y)
and,#	and,	and,x	and,y	and,(x)	and,(y)
asl,a	asl,	asl,x			
bcc,	bcs,	beq,	bmi,		
bne,	bpl,	bvc,	bvs,		
bit,	brk,				
clc,	cld,	cli,	clv,		
cmp,#	cmp,	cmp,x	cmp,y	cmp,(x)	cmp,(y)
cpx,#	cpx,	cpy,#	cpy,		
dec,	dec,x	dex,	dey,		
eor,#	eor,	eor,x	eor,y	eor,(x)	eor,(y)
inc,	inc,x	inx,	iny,		
jmp,	(jmp),	jsr,			
lda,#	lda,	lda,x	lda,y	lda,(x)	lda,(y)
ldx,#	ldx,		ldx,y		
ldy,#	ldy,	ldy,x			
lsr,a	lsr,	lsr,x			nop,
ora,#	ora,	ora,x	ora,y	ora,(x)	ora,(y)
pha,	php,	pla,	plp,		
rol,a	rol,	rol,x			
ror,a	ror,	ror,x			
rti,	rts,				
sbc,#	sbc,	sbc,x	sbc,y	sbc,(x)	sbc,(y)
sec,	sed,	sei,			
sta,		sta,x	sta,y	sta,(x)	sta,(y)
stx,	stx,y	sty,	sty,x		
tax,	tay,	tsx,	txa,	txs,	tya,

Appendix B: Memory Map

3 - \$3a

Parameter stack, **lsb** section.

\$3b - \$72

Parameter stack, **msb** section.

\$8b - \$8c

w (work area for code words).

\$8d - \$8e

w2 (work area for code words).

\$9e - \$9f

w3 (work area for code words).

\$100 - \$1ff

Return stack.

\$200 - \$258

Text input buffer.

\$33c - \$35a

find buffer.

\$35b - \$3d9

pad Scratch pad memory, Cassette Buffer, untouched by durexForth.

\$3da - \$3fb

#> buffer.

\$801 - here

Forth Kernel followed by code and data space.

latest - \$9fff

Dictionary. Grows downwards as needed. On the EasyFlash cartridge the dictionary stops at \$9cff instead, and \$9d00 - \$9f27 holds the driver that serves **include** from flash.

\$a000 - \$cbff

Editor text buffer.

\$cc00 - \$cfff

Hi-res graphics colors.

\$d000 - \$dfff

I/O area.

\$e000 - \$ffff

Kernal / hi-res graphics bitmap.

Appendix C: System Exceptions

The following exception numbers are used by durexForth.

- 1**
abort
- 2**
abort"
- 4**
stack underflow
- 8**
dictionary overflow (out of memory)
- 10**
division by zero
- 13**
undefined word
- 16**
attempt to use an empty string as a name
- 28**
user interrupt (triggered by `RESTORE` or `brk` instruction)
- 37**
file I/O exception

Appendix D: Word Anatomy

Let us define a word and see what it gets compiled to.

```
: bg $d020 c! ;
```

Information about the word is split into two areas of memory, the dictionary and the code/data space. Code and data are placed in an upward-growing segment starting at \$801, and the dictionary grows downward from **top**. **latest** points to the last dictionary record. A dictionary record consists of a counted string with flags, and an execution token (*xt*).

To inspect the dictionary entry, type **latest dump**. You should see something like this:

```
6228  02 42 47 fd 39 28 39 01 .bg.9(9.  
...
```

For this run, the name token of **bg** is placed at address \$6228. The first byte, **02**, is the name length (**bg** has two characters). After that, the string **bg** follows. (\$42 = **b**, \$47 = **g**). The final two bytes contain the execution token of **bg**, starting at \$39fd.

The name length byte is also used to store special attributes of the word. Bit 7 is "immediate" flag, which means that the word should execute immediately instead of being compiled into word definitions. (**(** is such an example of an immediate word that does not get compiled.) Bit 6 is the "no-tail-call-elimination" flag, which makes sure that tail call elimination (the practice of replacing **jsr**/**rts** with **jmp**) is not performed if this word is the **jsr** target. Since **bg** does not have these flags set, bits 7 and 6 are both clear.

We saw that the **bg** execution token is \$39fd. To inspect the code, type **\$39fd dump** or **latest >xt dump**.

The code section contains pure 6502 machine code.

```
39fd  20 15 11 20 d0 4c 0e 09  .. Pl..  
...
```

20 15 11

jsr \$1115. \$1115 is the address of the **lit** code. **lit** copies the two following bytes to parameter stack.

20 d0

\$d020. The parameter to the **lit** word. When executed, **lit** will add \$d020 to the parameter stack.

4c 0e 09

jmp \$90e. \$90e is the address of the **c!** code.

Appendix E: Avoiding Stack Crashes

Stack overflow and underflow are common causes for errors and crashes. Simply put, the data stack must not contain too many or too few items. This section describes some techniques to avoid such errors.

One helpful technique to avoid stack crashes is to add comments about stack usage. In this example, we imagine a graphics word "drawbox" that draws a black box. (color --) indicates that it takes one argument on stack, and on exit it should leave nothing on the stack. The comments inside the word (starting with £) indicate what the stack looks like after the line has executed.

```
: drawbox ( color -- )
10 begin dup 20 < while £ color x
10 begin dup 20 < while £ color x y
2dup £ color x y x y
4 pick £ color x y x y color
blkcol £ color x y
1+ repeat drop £ color x
1+ repeat 2drop ;
```

Once the word is working as supposed, it may be nice to again remove the comments, as they are no longer very interesting to read.

NOTE

The Forth standard defines backslash (\) as the line comment character, but the C64 lacks a real backslash. Moreover, ASCII \ and PETSCII £ both map to \$5c. Therefore, the £ character is used as a substitution on the C64.

Another useful technique during development is to check at the end of your main loop that the stack depth is what you expect it to. This will catch stack underflows and overflows.

```
: mainloop begin
( do stuff here... )
depth abort" depth not 0"
again ;
```

Appendix F: Internet Resources

Forth Books and Papers

- [Starting Forth](#)
- [Thinking Forth](#)
- [Moving Forth: a series on writing Forth kernels](#)
- [Forth Interest Group](#)
- [Blazin' Forth --- An inside look at the Blazin' Forth compiler](#)
- [The Evolution of FORTH, an unusual language](#)
- [A Beginner's Guide to Forth](#)

Other Forths

- [colorForth](#)
- [Gforth](#)
- [jonesforth](#)
- [PETTIL](#)
- [volksFORTH](#)

Forth Standards

- [Forth 2012 Standard](#)
- [ANS Forth](#)
- [FORTH-83 STANDARD](#)
- [FORTH-79 STANDARD](#)

C64 References

- [ACME Cross-Assembler](#)
- [Codebase 64](#)
- [All About Your C64](#)
- [Mapping the Commodore 64](#)

Appendix G: License

durexForth

Copyright (c) 2008 Johan Kotlinski, Mats Andrén

Copyright (c) 2012 Kevin Lee Reno

Copyright (c) 2017 Corey Minter

Copyright (c) 2020 Poindexter Frink, Richard Halkyard

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

/ MOD */MOD */ ?NEGATE ?DNEGATE DABS SM/REM
Forth Interest Group UK. Public domain.

UM/MOD

Copyright (c) 2002 Garth Wilson. Public domain.

MOVE

Based on cc65 memmove.s

This software is provided 'as-is', without any express or implied warranty. In no event will the authors be held liable for any damages arising from the use of this software.

Permission is granted to anyone to use this software for any purpose, including commercial applications, and to alter it and redistribute it freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not claim that you wrote the original software. If you use this software in a product, an acknowledgment in the product documentation would be

appreciated but is not required.

2. Altered source versions must be plainly marked as such, and must not be misrepresented as being the original software.
3. This notice may not be removed or altered from any source distribution.

Forth Test Suite

(C) 1995 JOHNS HOPKINS UNIVERSITY / APPLIED PHYSICS LABORATORY
MAY BE DISTRIBUTED FREELY AS LONG AS THIS COPYRIGHT NOTICE REMAINS.
VERSION 1.2

This program was written by Gerry Jackson in 2007, with contributions from others where indicated, and is in the public domain - it can be distributed and/or modified in any way but please retain this notice.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Word Index

@

!, 20, 23

#, 28

#>, 29

#s, 28

', 30

*, 15

*/, 15

*/mod, 16

+, 15

+!, 15

+branch, 68

+loop, 24

., 21

-, 15

-branch, 68

-leading, 27

-rot, 13

-trailing, 27

., 14

.", 26

.(, 26

.s, 14

/, 15

/mod, 15

/string, 26

0<, 16

0<>, 19

0=, 19

0>, 38

1+, 15

1-, 15

2!, 38

2*, 15

2+, 15

2/, 15

2@, 37

2drop, 13

2dup, 13

2over, 38

2swap, 38

;;, 20

:+, 68

:-, 68

:noname, 20

;;, 21

<, 16

<#, 28

<>, 20

=, 19

>, 16

>body, 38

>in, 25

>lower, 28

>number, 38

>r, 13

>upper, 28

>xt, 30

?bands, 49

?dup, 13

?irq, 47

?raster, 48

@, 20, 23

@:, 68

@@, 68

[, 21

['], 21

[char], 21

\, 14

], 21

□□, 14

£, 14

A

abort, 31, 75

abort", 31, 75

abs, 16

accept, 25

acptr, 36

again, 24

align, 37

aligned, 37

allot, 21

and, 20

ar, 38

at-xy, 26

B

back, 40

band+, 49

band-bg!, 50
band-border!, 50
band-charset!, 50
band-screen!, 50
band-sync, 49
band-xscroll!, 50
band-yscroll!, 50
bands-clear, 49
bands-off, 49
bands-on, 49
base, 16
begin, 24
bg-play, 51
bg-stop, 51
bl, 14
blit-and, 42
blit-blk, 42
blit-col!, 42
blit-get, 42
blit-op, 42
blit-or, 42
blit-xor, 42
blkcol, 41
bsave, 37

C

c!, 20
c,, 21
c@, 20
cam!, 59
cam+, 60
cam-start, 60
cam-stop, 60
cam-x, 60
cam-y, 60
case, 24
catch, 30
cell+, 37
cells, 37
char, 25
char+, 37
char-data, 56
chardef, 56
charrom, 56
charrom-lc, 56
charrom>, 56
chars, 37
charset!, 55

charset-lock, 55
charset-unlock, 55
charset@, 55
chkin, 35
chkout, 35
chrin, 35
ciout, 36
circle, 41
clear-disk, 33
close, 35
clrchn, 35
clrcol, 41
code, 21
col-at, 56
col-scroll, 58
col38, 57
col40, 57
compare, 27
compile,, 22
constant, 23
control!, 44
copy, 33
count, 26
cpy-and, 42
cpy-blk, 42
cpy-or, 42
cpy-xor, 42
create, 22
cstr, 28
cutoff!, 44

D

d!, 17
d+, 37
d., 17
d0<, 17
d0<>, 17
d0=, 17
d0>, 17
d<, 17
d<>, 17
d=, 17
d>, 17
d@, 17
dabs, 18
decimal, 16
defcode, 22
define, 22

depth, 14
device, 32
di, 47
dir, 37
dnegate, 18
do, 24
dos, 32, 37
dowords, 23
drawchar, 41
drop, 13
du<, 17
du>, 17
dump, 31
dup, 13

E

ei, 47
else, 24
emit, 14
end-code, 21
endcase, 24
endof, 24
env3@, 44
environment?, 37
erase, 20
evaluate, 26
execute, 30
exit, 24

F

fl, 19
f*, 18
f+, 18
f-, 18
f., 19
f/, 18
f0<, 18
f0<>, 18
f0=, 18
f0>, 18
f<, 19
f<>, 19
f=, 19
f>, 19
f@, 19
false, 38
fappend, 33
fclose, 34

fcreate, 33
fdepth?, 19
fdrop, 19
fdup, 19
fgetc, 34
fill, 20
filter!, 44
find, 30
find-name, 30
flip, 43
fm/mod, 18
fopen, 33
forward, 40
fputc, 33
fread-type, 34
freq!, 44
fwrite, 34

H

header, 22
here, 22
hex, 17
hide, 22
hires, 40
hold, 29

I

i, 24
if, 23
immediate, 21
include, 31
included, 31
index-of, 27
init, 40
initialize, 33
invert, 20
ioabort, 35
irq!, 47
irq-off, 47

J

j, 24
joy-down?, 51
joy-fire?, 51
joy-left?, 51
joy-right?, 51
joy-up?, 51
joy1, 51

joy2, [51](#)

K

kb-key, [54](#)

kb?, [54](#)

key, [25](#)

key?, [25](#)

L

latest, [22](#)

leave, [24](#)

left, [27](#), [40](#)

line, [41](#)

listen, [36](#)

literal, [21](#)

loadb, [32](#), [62](#)

loop, [24](#)

lores, [41](#)

lower, [28](#)

ls, [32](#)

lsb, [14](#), [67](#)

lshift, [16](#)

M

m*, [18](#)

m+, [18](#)

map!, [59](#)

map-coll!, [59](#)

marker, [22](#)

max, [16](#)

mid, [27](#)

min, [16](#)

mir, [43](#)

mod, [15](#)

mouse-init, [52](#)

mouse-lb?, [52](#)

mouse-port, [52](#)

mouse-rb?, [52](#)

mouse-speed, [52](#)

mouse-update, [52](#)

mouse-x, [52](#)

mouse-y, [52](#)

move, [20](#)

moveto, [40](#)

msb, [14](#), [67](#)

N

n, [31](#)

negate, [16](#)

newdisk, [33](#)

nip, [13](#)

note!, [44](#)

O

of, [24](#)

open, [35](#)

or, [20](#)

osc3@, [44](#)

over, [13](#)

P

p>s, [57](#)

pad, [26](#)

page, [15](#)

paint, [41](#)

parse, [25](#)

parse-name, [25](#)

peek, [41](#)

pen, [41](#)

pendown, [40](#)

penup, [40](#)

pick, [13](#)

play-mml, [46](#)

plot, [41](#)

postpone, [22](#)

pulse!, [44](#)

Q

quit, [26](#)

R

r>, [13](#)

r@, [14](#)

raster!, [48](#)

raster-off, [48](#)

rderr, [32](#)

rdir, [32](#)

readst, [35](#)

record, [34](#)

recurse, [24](#)

refill, [25](#)

rel-close, [34](#)

rel-create, [34](#)

rel-open, [34](#)

rename, [33](#)

repeat, [24](#)

require, 31
required, 31
right, 27, 40
romfs driver, 65
rot, 13
row24, 58
row25, 58
rshift, 16
rvs, 15

S

s", 26
s>d, 18
s>f, 18
save-forth, 32
save-pack, 39
save-prg, 39
saveb, 32
scr-at, 56
scratch, 32
screen!, 55
screen@, 55
scroll-down, 58
scroll-left, 58
scroll-right, 58
scroll-up, 58
search, 27
second, 36
see, 31
send-cmd, 32, 37
sid!, 63
sid-init, 63
sid-load, 63
sid-off, 63
sid-on, 63
sign, 29
size, 31
sm/rem, 38
source, 25
source-id, 25
sp-behind, 53
sp-bg-coll, 53
sp-front, 53
sp-hit?, 53
sp-mc-off, 53
sp-mc-on, 53
sp-mc0!, 53
sp-mc1!, 53

sp-ptr!, 53
sp-sp-coll, 53
sp-x@, 53
sp-y@, 53
space, 14
spaces, 14
spin, 43
split, 16
sr, 38
srاد!, 44
starts-with?, 27
state, 22
status, 33
str=, 27
swap, 13
synonym, 21
sys, 38

T

talk, 36
text, 41
text!, 57
then, 23
throw, 30
tile!, 57
tile-coll!, 57
tksa, 36
to, 23
top, 38
top!, 38
trim, 27
true, 38
tuck, 13
turtle!, 40
turtle@, 40
type, 26

U

u., 14
u<, 16
u>, 16
ud., 17
um*, 18
um/mod, 18
unlisten, 36
unloop, 24
untalk, 36
until, 24

upper, [28](#)

V

validate, [33](#)

value, [23](#)

variable, [23](#)

vic-bank!, [55](#)

vic-bank@, [55](#)

vic-base, [55](#)

voice!, [44](#)

volume!, [44](#)

W

w, [67](#)

w2, [67](#)

w3, [67](#)

while, [24](#)

win!, [59](#)

within, [16](#)

word, [25](#)

words, [31](#)

wrap-down, [58](#)

wrap-left, [58](#)

wrap-right, [58](#)

wrap-up, [58](#)

X

xor, [20](#)

xpandx, [43](#)

xpandy, [43](#)

xr, [38](#)

xscroll!, [57](#)

xscroll@, [57](#)

xy@, [26](#)

Y

yr, [38](#)

yscroll!, [57](#)

yscroll@, [57](#)